Universidad Tecnológica de La Habana
"José Antonio Echeverría"
Facultad de Ingeniería Informática



Informe de Práctica Profesional I

Implementación de nuevos algoritmos de agrupamiento en BHAVRP

Autor:

Maykel Kevin Díaz Menéndez

Tutores:

Ing. Eric Ramos Aragón
Ing. Ananda de la Caridad Morales Morales

La Habana, Cuba

Marzo 2026

Resumen

El Problema de Planificación de Rutas de Vehículos con Múltiples Depósitos (MDVRP) constituye una variante compleja dentro de los problemas de optimización combinatoria. En la práctica, este problema suele resolverse dividiéndolo en dos etapas: asignar los clientes a los depósitos y construir las rutas desde cada depósito. La Biblioteca de Heurísticas de Asignación para Problemas de Planificación de Rutas de Vehículos con Múltiples Depósitos (BHAVRP) es un componente de software diseñado para resolver esta primera fase de asignación. La biblioteca presenta una arquitectura modular que encapsula métodos de agrupamiento clasificados en particionales y jerárquicos, incluye un sistema de tratamiento de excepciones y un conjunto de métricas de evaluación internas. Actualmente, BHAVRP carece de algoritmos de agrupamiento basados en densidad y cuenta con un solo algoritmo de agrupamiento jerárquico. Se propone la incorporación de dos nuevos algoritmos de agrupamiento: DBSCAN como basado en densidad, y CURE como jerárquico. Ambos algoritmos de agrupamiento son adaptados para resolver la fase de asignación de clientes a depósitos incorporando restricciones de capacidad de los depósitos y mecanismos de asignación controlada. Por último, se realiza un estudio experimental que permite garantizar el correcto funcionamiento de los módulos implementados, así como identificar qué algoritmos generan asignaciones de mayor calidad. Para ello, se utilizan los criterios definidos por las métricas de evaluación que ya tiene BHAVRP.

Palabras clave: algoritmos de agrupamiento, BHAVRP, CURE, DBSCAN, MDVRP.

Abstract

The Multi-Depot Vehicle Routing Problem (MDVRP) is a complex variant within combinatorial optimization problems. In practice, this problem is often solved by dividing it into two stages: assigning customers to depots and building routes from each depot. The Assignment Heuristics Library for Multi-Depot Vehicle Routing Problems (BHAVRP) is a software component designed to solve this first assignment phase. The library features a modular architecture that encapsulates clustering methods classified into partitional and hierarchical, includes an exception handling system, and a set of internal evaluation metrics. Currently, BHAVRP lacks density-based clustering algorithms and has only one hierarchical clustering algorithm. The incorporation of two new clustering algorithms is proposed: DBSCAN as a density-based algorithm, and CURE as a hierarchical algorithm. Both clustering algorithms are adapted to solve the customer-to-depot assignment phase by incorporating depot capacity constraints and controlled assignment mechanisms. Finally, an experimental study is conducted to ensure the correct functioning of the implemented modules, as well as to identify which algorithms generate higher-quality assignments. For this purpose, the criteria defined by the evaluation metrics already present in BHAVRP are used.

Keywords: BHAVRP, CURE, clustering algorithms, DBSCAN, MDVRP.

Índice general

Introducción **1**

1. Fundamentos teóricos del problema de planificación de rutas y su solución mediante diferentes métodos **5**

1.1. Problemas de Planificación de Rutas de Vehículos 5

1.1.1. Problema de Planificación de Rutas de Vehículos con Múltiples Depósitos 6

1.2. Algoritmos heurísticos 8

1.3. Algoritmos de agrupamiento para la fase de asignación de clientes a depósitos . 11

1.4. Biblioteca de Heurísticas de Asignación en la Planificación de Rutas de Vehículos con Múltiples Depósitos 14

1.5. Conclusiones parciales 19

2. Incorporación de algoritmos a la Biblioteca de Heurísticas de Asignación del Problema de Planificación de Rutas con Múltiples Depósitos **20**

2.1. Algoritmo DBSCAN 20

2.2. Algoritmo CURE 23

2.3. Comparación de los algoritmos de agrupamiento 26

2.4. Incorporación de los algoritmos propuestos a BHAVRP 27

2.5. Conclusiones parciales 31

3. Análisis experimental **32**

3.1. Pruebas funcionales 32

3.1.1. Prueba del método *distance* en DBSCAN 32

3.1.2. Pruebas del constructor y método merge de la clase CureCluster 33

3.1.3. Pruebas del método *calculateClusterDistance* de la clase CURE 34

3.2. Pruebas unitarias 35

3.3. Comparación entre los algoritmos de agrupamiento 37

3.3.1. Análisis estadístico 39

3.4. Comparación entre los algoritmos de agrupamiento jerárquico 42

3.5. Conclusiones parciales 44

Conclusiones **46**

Recomendaciones **47**

Referencias bibliográficas **47**

Índice de figuras

1.1. Representación gráfica de MDVRP. 8
1.2. Técnicas utilizadas para la solución de MDVRP. 9
1.3. Representación gráfica de algoritmos de agrupamiento particionales [43]. . . . 13
1.4. Representación gráfica de algoritmos de agrupamiento jerárquicos [43]. 13
1.5. Representación gráfica de algoritmos de agrupamiento basados en densidad [43]. 14
1.6. Diagrama de actividades del flujo de BHAVRP. 18
1.7. Diagrama de arquitectura n-capas con enfoque en reutilización de BHAVRP. . . 18

2.1. Ejemplo de la agrupación de puntos por densidad de DBSCAN. 21
2.2. Diagrama de flujo del algoritmo DBSCAN. 22
2.3. Diagrama de flujo del algoritmo DBSCAN a MDVRP. 22
2.4. Flujo de CURE. 23
2.5. Diagrama de flujo del algoritmo CURE. 25
2.6. Diagrama de flujo del algoritmo CURE adaptado al MDVRP. 26
2.7. Diagrama de arquitectura actualizado. 29
2.8. Diagrama de clases actualizado. 31

3.1. Resultados de las pruebas del método distance. 33
3.2. Resultados de las pruebas de constructor y del método merge. 34
3.3. Resultados de las pruebas del método calculateClusterDistance. 35

Índice de tablas

1.1. Comparación entre las herramientas que resuelven el MDVRP. 17

2.1. Comparación de algoritmos de BHAVRP con los propuestos. 27
2.2. Clases implementadas para la incorporación de los nuevos algoritmos de agrupamiento. 28

3.1. Pruebas unitarias de caja negra del algoritmo DBSCAN. 36
3.2. Pruebas unitarias de caja negra del algoritmo CURE. 37
3.3. Descripción de las instancias. 38
3.4. Resumen de las métricas incorporadas en BHAVRP. 39
3.5. Resultados de la prueba de Friedman NxN. 40
3.6. Resumen análisis post-hoc. 41
3.7. Descripción de las nuevas instancias. 43
3.8. Resumen de los resultados de Wilcoxon. 43

Introducción

La gestión de la distribución, el transporte y la planificación de rutas son temas de gran importancia que han sido abordados a nivel internacional. Son ramas de la logística donde yacen numerosas oportunidades de mejora y por esta razón ha recibido especial atención motivando un intenso trabajo teórico en el desarrollo de modelos y algoritmos. En este contexto surgen los Problemas de Planificación de Rutas de Vehículos (*Vehicle Routing Problems*, VRP) [1].

En [2] se introduce el *"Truck Dispatching Problem"*, modelando cómo una flota de camiones homogéneos podría satisfacer la demanda de petróleo de varias estaciones de servicio desde un depósito central y con una distancia mínima recorrida. Cinco años después, en [3] se generaliza este a un problema de optimización lineal que se encuentra comúnmente en el ámbito de la logística y el transporte. El VRP es un problema NP-duro [4] por lo que se consideran muy difíciles de resolver. Los problemas de la vida real son considerablemente más grandes y complejos que los escenarios académicos, por lo que las heurísticas y metaheurísticas son necesarias para obtener buenos resultados. Estos algoritmos no garantizan encontrar la solución óptima, pero sí proporcionan tiempos computacionales razonables. La cantidad de métodos de solución introducidos en la literatura académica ha crecido en las últimas décadas [5].

El Problema de Planificación de Rutas de Vehículos con Múltiples Depósitos (*Multi-Depot Vehicle Routing Problems*, MDVRP) es una variante del VRP clásico que incorpora varios depósitos con una ubicación predefinida. Cada depósito cuenta con una flota de vehículos de capacidad homogénea que tiene como objetivo satisfacer la demanda de un conjunto de clientes dispersos geográficamente. Cada vehículo debe comenzar y terminar su ruta en el mismo depósito, y cada cliente debe ser visitado una única vez. Teniendo en cuenta la distancia entre la ubicación de los clientes y depósitos, el MDVRP busca asignar un cliente a cada depósito y se construye una ruta para cada vehículo dentro de un depósito para atender dichos clientes. El objetivo al crear la ruta es reducir al mínimo el costo total del recorrido [6].

Durante un largo período de tiempo, los métodos heurísticos basados en la búsqueda tabú han sido los más empleados en la literatura [6]. Sin embargo, este problema genera una gran cantidad de combinaciones que se refleja en los altos tiempos computacionales para obtener la solución. El potencial de las técnicas de análisis de clústeres para la solución de los MDVRP ha sido reconocido por autores como [7], [2], [8] y [9]. El análisis de clústeres estudia la división de objetos en grupos basado en una o varias características. Un grupo se define como una zona con muchos objetos parecidos muy cercanos entre sí, que está separada de otros grupos por zonas donde casi no hay objetos. Esta definición de grupo es bastante acorde a la agrupación de clientes por lo que se propone usar el análisis de clústeres en la solución del MDVRP, específicamente en la fase de asignación de clientes a depósitos [7]. En [10] se describe la técnica "divide y

vencerás", utilizada para reducir la complejidad computacional en el empleo heurísticas para resolver el MDVRP.

Sin embargo, aún existen desafíos en cuanto a la eficiencia computacional debido a los altos tiempos y la calidad de las agrupaciones dependiendo del escenario [7]. En la literatura sobresalen algoritmos como *Density-Based Spatial Clustering of Applications with Noise* (DBSCAN) que permiten identificar clústeres basados en la densidad local [11]. Por otra parte, *Clustering Using Representatives* (CURE) identifica variaciones en la forma de los clústeres mediante el uso de puntos representativos [12].

En este contexto, la Universidad Tecnológica de la Habana "José Antonio Echeverría" (CUJAE) ha desarrollado la Biblioteca de Heurísticas de Asignación para Problemas de Planificación de Rutas de Vehículos con Múltiples Depósitos (BHAVRP). Este componente está diseñado para resolver la fase de asignación en MDVRP utilizando una arquitectura modular y 20 algoritmos repartidos en heurísticas clásicas de asignación y algoritmos de agrupamiento de Minería de Datos adaptados al contexto de los MDVRP [13]. Esta alternativa gratuita y de código abierto supone una ventaja frente a otras herramientas que no tienen una colección de algoritmos tan extensa o son software comercial.

Esta biblioteca, en la categoría de algoritmos de agrupamiento, incluye algoritmos particionales, algoritmos jerárquicos y un algoritmo basado en densidad. En la literatura se ha sugerido que ciertas métricas de validación internas como Dunn Index o Silhouette podrían tener una correlación con la calidad de las rutas en la posterior fase de planificación. En investigaciones anteriores [14], [15], se realizó un análisis experimental donde se concluyó que no había una correlación positiva significativa entre las métricas internas dentro de BHAVRP y la calidad de las rutas obtenidas. Sin embargo, este estudio fue realizado con poca variedad de algoritmos de agrupamiento y sin considerar una amplia variedad de heurísticas de construcción.

Debido a la situación problemática antes descrita, se identifica como problema a resolver la falta de un estudio exhaustivo que determine si existe alguna métrica interna que correlacione positivamente con la calidad de las rutas obtenidas en la planificación, lo que impide establecer un criterio de selección de los algoritmos de BHAVRP.

Para dar solución al problema planteado se traza como objetivo general: Desarrollar una nueva versión de BHAVRP que incluya más algoritmos de agrupamiento basados en densidad.

Para dar cumplimiento al objetivo general se definieron los siguientes objetivos específicos con sus respectivas tareas:

1. Identificar algoritmos de agrupamiento utilizados para resolver la fase de asignación de los problemas de planificación de rutas de vehículos con múltiples depósitos.

    - Realizar búsqueda bibliográfica de las características de los problemas de planifica-

ción de rutas de vehículos con múltiples depósitos y sus aplicaciones prácticas.

- Realizar búsqueda bibliográfica de las propuestas de solución de MDVRP donde se realice la fase de asignación de clientes a depósitos usando como vía de resolución los algoritmos de agrupamiento.

2. Implementar en BHAVRP un nuevo algoritmo de agrupamiento.

   - Analizar algoritmos de agrupamiento en cuanto a su funcionamiento para su incorporación en la nueva versión de la biblioteca.
   - Seleccionar los algoritmos de agrupamiento que se van a incorporar a la biblioteca para resolver la variante MDVRP.
   - Implementar y adaptar el algoritmo seleccionado teniendo en cuenta las características que presentan.

3. Validar el funcionamiento de estos algoritmos en BHAVRP mediante la ejecución de experimentos.

   - Realizar pruebas unitarias, donde se muestre que el algoritmo implementado funcione correctamente.
   - Comparar los resultados obtenidos por los algoritmos de agrupamiento propuestos con los algoritmos de agrupamiento implementados en la versión anterior de la biblioteca.

El cumplimiento del objetivo general tiene como valor práctico un aporte significativo en la resolución de la fase de asignación del problema de planificación de rutas de vehículos con múltiples depósitos. Los resultados tienen valor científico y tecnológico ya que se pone a disposición de la comunidad científica un componente de software que brinda nuevos algoritmos para la solución de MDVRP.

Para alcanzar los objetivos mencionados, el trabajo se encuentra estructurado en tres capítulos, incluyendo secciones importantes como conclusiones, recomendaciones y referencias bibliográficas. El Capítulo 1: Fundamentos teóricos del problema de planificación de rutas y su solución mediante diferentes métodos donde se presenta el estado del arte del MDVRP, las estrategias de solución existentes, así como un análisis comparativo de bibliotecas. El Capítulo 2: Incorporación de algoritmos a la Biblioteca de Heurísticas de Asignación del Problema de Planificación de Rutas con Múltiples Depósitos describe los algoritmos de agrupamiento de BHAVRP, los algoritmos propuestos detallando el funcionamiento de estos según la literatura y las adaptaciones al MDVRP que hay que realizar. Finalmente, el Capítulo 3: Análisis experimental

expone varias pruebas unitarias de las funciones más relevantes de los algoritmos implementados, así como un estudio comparativo entre algoritmos de asignación aplicados sobre ciertas instancias y pruebas estadísticas no paramétricas para determinar su rendimiento y aplicabilidad.

Capítulo 1: Fundamentos teóricos del problema de planificación de rutas y su solución mediante diferentes métodos

En este capítulo se describen los aspectos teóricos de la investigación que se encuentran relacionados con los MDVRP. Entre los aspectos que se abordan están las características y variantes del problema, así como los diferentes métodos de solución encontrados en la literatura. Además, se hace énfasis en los algoritmos de agrupamiento como alternativa para la fase de asignación de clientes a depósitos. Por último, se describe la Biblioteca de Heurísticas de Asignación para Problemas de Planificación de Rutas de Vehículos con Múltiples Depósitos y sus algoritmos existentes.

1.1 Problemas de Planificación de Rutas de Vehículos

El concepto de Problema de Planificación de Rutas de Vehículos (*Vehicle Routing Problem*, VRP) es una generalización del Problema del Viajero (*Traveling Salesman Problem*, TSP). En concreto, el VRP es un problema de optimización combinatoria cuyo objetivo es atender las demandas de clientes que se encuentran dispersos geográficamente sujetos a un conjunto de restricciones. La diferencia radica en que TSP dispone de un mapa con múltiples puntos que un individuo tiene que visitar para volver a su punto de origen realizando la ruta más corta posible. En cambio, el VRP no se limita a solo una persona, sino que pueden existir más de un individuo, en este caso vehículos, que normalmente deben entregar objetos en diversos puntos de la ciudad a clientes. De este modo, el problema se vuelve más complejo que el TSP [16].

Las aplicaciones informáticas para el diseño de rutas se derivan de modelos VRP que atienden diferentes necesidades e incluyen diversas restricciones. Esto ha dado como resultado el surgimiento de diferentes variantes del clásico VRP, de las cuales se pueden mencionar las siguientes:

- VRP con recolección y entrega (VRPPD): estudia el caso donde una empresa debe recolectar y entregar bienes en cantidades específicas para cada cliente visitado [17].
- VRP con flota heterogénea: los vehículos tienen diferentes capacidades de carga [18].
- *Open VRP* (OVRP): se refiere al caso de cuando el vehículo no regresa al depósito de origen, es decir, que se dirige a otro depósito o sitio diferente para cargar productos, en un contexto de flota rentada [19].
- VRP con ventanas de tiempo (VRPTW): plantea que cada cliente tiene que ser atendido de manera obligada dentro de un cierto horario específico [20].

- VRP con depósitos múltiples (MDVRP): implica que la empresa posee diversos depósitos desde los cuales puede abastecer a los clientes [21].
- *Stochastic VRP* (SVRP): Propone que las variables involucradas son desconocidas o aleatorias, e incluso que se pueden encontrar en un rango de probabilidad, tales como el número de clientes, sus demandas, tiempo de servicio, tiempo de viaje, entre otras [22].
- *Periodic VRP* (PVRP): analiza un problema donde los pedidos pueden ser llevados sólo en ciertos días [23].
- Capacitated VRP: estudia el caso donde los vehículos tienen una capacidad máxima de carga y se deben diseñar rutas que satisfagan la demanda de los clientes sin exceder dicha capacidad [24].

1.1.1 Problema de Planificación de Rutas de Vehículos con Múltiples Depósitos

El MDVRP es un problema de optimización combinatoria, categorizado por su complejidad como NP-Duro [25]. Por lo que es difícil encontrar su solución óptima usando métodos exactos en un tiempo polinomial. Esta variante cuenta con varios depósitos y cada uno tiene su propia flota de vehículos [26].

El problema puede ser representado mediante modelos de optimización. Sea el grafo completo $G = (V, A)$, donde el conjunto de vértices $V = \{v_{01}, ..., v_{0d}\} \cup \{v_1, ..., v_n\}$ incluye los d depósitos y c clientes y el conjunto de arcos $A = \{(i, j) \vee i, j \in V, i \neq j\}$ [27]. El objetivo es minimizar el costo total de las rutas, sujeto a las restricciones operativas. La función objetivo, variables de decisión y las restricciones fundamentales son [28]:

Función Objetivo: Minimizar el costo total de transporte, generalmente expresado como la suma de las distancias recorridas por todos los vehículos.

$$\text{mín} \sum_{k \in K} \sum_{(i,j) \in A} c_{ij} \cdot x_{ijk} \tag{1.1}$$

Variables de Decisión:

$$x_{ijk} = \begin{cases} 1 & \text{si el vehículo } k \text{ viaja del vértice } i \text{ al vértice } j \\ 0 & \text{en caso contrario} \end{cases} \qquad \forall (i, j) \in A,\ \forall k \in K \tag{1.2}$$

Restricciones:

Cada ruta debe comenzar y terminar en el mismo depósito.

$$\text{Salida del Depósito:} \quad \sum_{j \in C} x_{djk} = 1 \quad \forall d \in V_0,\ \forall k \in K \tag{1.3}$$

$$\text{Entrada del Depósito:} \quad \sum_{j \in C} x_{jdk} = 1 \quad \forall d \in V_0,\ \forall k \in K \tag{1.4}$$

- Cada cliente debe ser visitado exactamente una vez.

$$\sum_{k \in K} \sum_{j \in V, j \neq i} x_{ijk} = 1 \quad \forall i \in C \tag{1.5}$$

- La demanda del cliente es un número diferente a cero y no negativo.

$$q_i > 0 \quad \forall i \in C \tag{1.6}$$

- La demanda total de cada ruta no debe exceder la capacidad del vehículo.

$$\sum_{i \in C} q_i \cdot \left(\sum_{j \in V, j \neq i} x_{ijk} \right) \leq Q \quad \forall k \in K \tag{1.7}$$

- Todos los vehículos tienen una misma capacidad.

$$Q_k = Q \quad \forall k \in K \tag{1.8}$$

- Se dispone de un número limitado de vehículos.

$$\sum_{k \in K} \sum_{j \in C} x_{djk} \leq m_d \quad \forall d \in V_0 \tag{1.9}$$

- Todos los depósitos tienen el mismo número de vehículos disponibles.

$$m_d = m \quad \forall d \in V_0 \qquad \sum_{k \in K} \sum_{j \in C} x_{djk} \leq m \quad \forall d \in V_0 \tag{1.10}$$

En la Figura 1.1 se ilustra la estructura del problema:

- Se observan tres depósitos (representados por cuadrados), cada uno con una flota de vehículos.
- Los clientes (círculos rojos) son asignados a un depósito específico.
- Las líneas verdes muestran las rutas de los vehículos, que parten y regresan a su depósito de origen.

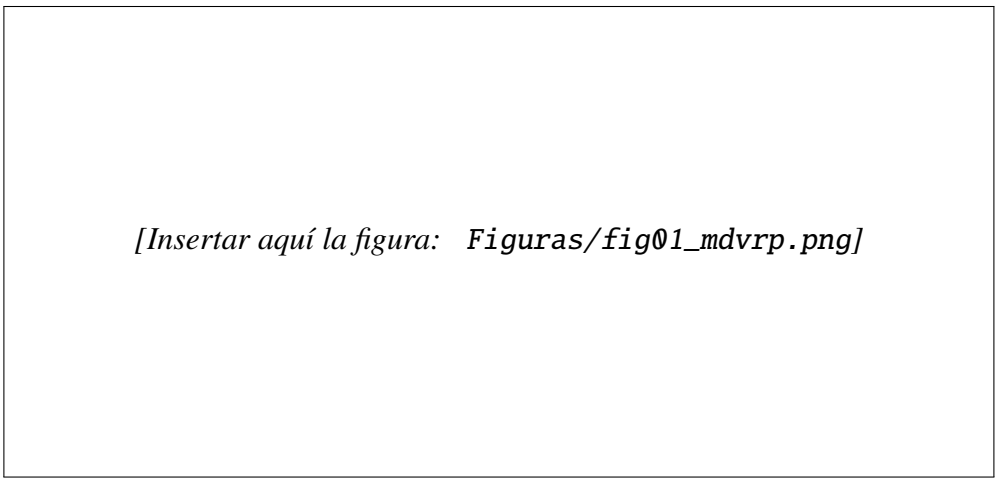


Figura 1.1: Representación gráfica de MDVRP.

La literatura documenta múltiples aplicaciones del MDVRP a problemas logísticos reales. Un estudio aplicado a una red de distribución de productos de limpieza y cuidado personal en Turquía utiliza el MDVRP para optimizar las rutas desde tres depósitos a 502 clientes. Mediante un enfoque de agrupar primero y rutear después se logran mejoras significativas en la eficiencia de la distribución, utilizando datos reales de demanda y coordenadas geográficas [29].

Otro ejemplo se encuentra en el Reino Unido, donde se desarrolla un modelo de programación entera mixta (MIP) [30] para resolver un MDVRP que incluye complejidades del mundo real como una flota heterogénea, costos de conductores y ventanas de tiempo. La implementación de este modelo permite reducir sustancialmente los costos de transporte sin necesidad de abrir nuevas instalaciones [31].

También se encuentra un caso de estudio en Estambul que trata sobre la distribución de una empresa de bebidas utilizando datos históricos de GPS para construir una matriz de tiempo-distancia más precisa. Se evalúan múltiples enfoques de optimización, incluyendo algoritmos evolutivos y de aprendizaje profundo, para resolver el MDVRP. Los resultados muestran que estos métodos son efectivos para manejar la complejidad y la naturaleza dinámica de las entregas en entornos urbanos de alta densidad [32].

1.2 Algoritmos heurísticos

Debido a la complejidad que presentan los VRP, existen dos tipos de algoritmos que pueden resolver este tipo de problemas, los métodos exactos y aproximados. Cada uno de ellos posee diferentes características que pueden considerarse ventajas y/o desventajas que deben ser tomadas en cuenta al ser utilizados [26]. En la Figura 1.2 se puede apreciar una taxonomía de estos métodos:

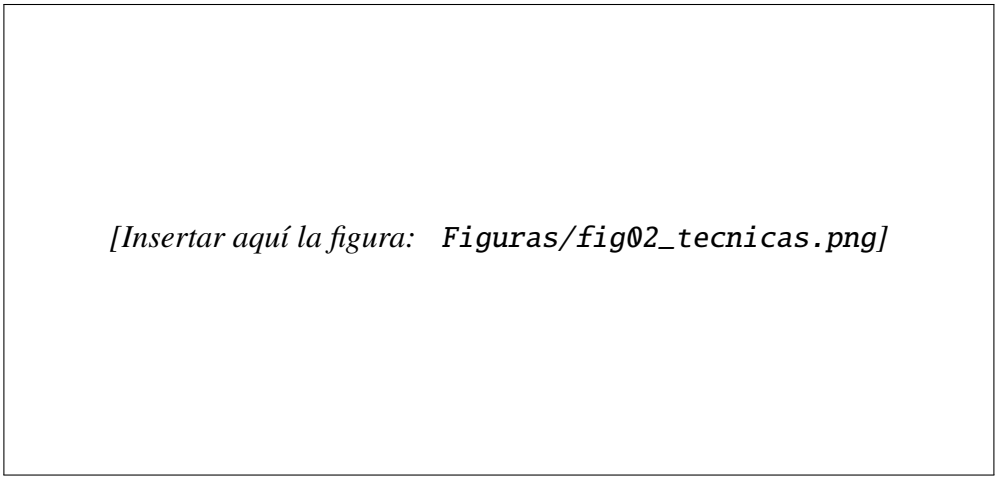


Figura 1.2: Técnicas utilizadas para la solución de MDVRP.

Entre los métodos exactos se pueden distinguir tres grupos: búsqueda directa de árbol, programación dinámica y programación lineal, entera y mixta. Todos estos métodos son eficientes en problemas de hasta 50 clientes, debido a restricciones de tiempo computacional. Se han llevado a cabo algunas investigaciones introduciendo métodos exactos para MDVRP [15], [26]. Algunos ejemplos de los más utilizados para el MDVRP de este tipo de métodos son:

- Ramificación y Corte: los algoritmos *Branch & Cut* se basan en el cálculo simultáneo de cotas inferiores y superiores de la solución óptima. Las cotas superiores se obtienen por medio de la resolución de las relajaciones lineales asociadas a cada nodo del árbol de enumeración, eventualmente ajustados por medio de planos de corte, mientras que las cotas inferiores son halladas por medio de procedimientos heurísticos y cuando la relajación lineal asociada con algún subproblema tiene óptimo entero [33].

- Programación Lineal: la programación lineal es un método a través del cual se optimiza una función objetivo, bien sea maximizando o minimizando dicha función, en la cual las variables están elevadas a la potencia 1. Este campo de la programación lineal es un área de la programación matemática, dedicada a maximizar o minimizar una función lineal que recibe el nombre de "función objetivo", de manera que las variables de tal función estén sujetas a una serie de restricciones expresadas a través de un sistema de ecuaciones o inecuaciones lineales [26].

La investigación se centra en los métodos heurísticos, los cuales se han caracterizado por ser procedimientos que garantizan buenas soluciones mediante una exploración limitada del espacio de búsqueda. Los métodos heurísticos se pueden clasificar en métodos constructivos y heurísticas de mejora [15], [26]. Algunas heurísticas utilizadas en la literatura para solucionar el MDVRP son:

- Heurística ad hoc: en 1993 fue presentada una nueva heurística ad hoc en el aporte de [34], su método de solución asigna los clientes a los depósitos de manera fácil y rápida. En el procedimiento de mejora, permite que la distancia total recorrida por la flota aumente con la esperanza de encontrar finalmente rutas muy cortas [26].

- Algoritmo de Ahorros: extensión del método clásico para VRP al contexto de múltiples depósitos. Asigna clientes a depósitos y luego construye rutas fusionando recorridos según el ahorro de distancia [3]. En [35] se presentó la primera adaptación para MDVRP.

- Algoritmo de Barrido: método de dos fases: primero agrupa clientes por ángulo polar respecto al depósito, luego construye rutas para cada grupo. Bueno para problemas de tamaño medio-grande [36].

- *Merge-Head and Drop-Tail*: una técnica planteada para resolver el MDVRP es la denominada estrategia *Merge-Head and Drop-Tail* (MHDT), combinada con optimización de combinación de sectores (SCO), aplicada por [37]; aquí se plantea un comportamiento inverso a la entrega, es decir debe generar rutas de recolección, dada esta única diferencia, el resto de las características y restricciones son idénticas, su propuesta funciona de igual manera para el MDVRP. En este método, primero asignan puntos de recolección de acuerdo con la distancia más cercana a los puntos de recolección, luego cada planta resuelve el VRP respectivamente, asignando clientes a los vehículos y planificando el orden en que los vehículos visitan a los clientes. En el último paso, proponen el algoritmo de optimización de combinación de sectores (SCO) para generar múltiples soluciones iniciales, y luego estas soluciones iniciales se mejoran utilizando la estrategia de combinación de cabeza y caída (MHDT).

Las metaheurísticas son estrategias de alto nivel que guían y modifican heurísticas subyacentes para explorar el espacio de búsqueda de forma eficiente. Estos algoritmos se basan en la exploración iterativa del espacio de soluciones, utilizando estrategias para escapar de los óptimos locales. Estas se clasifican en basadas en población o basadas en un punto [38].

La Búsqueda Tabú se mantiene como uno de los mejores enfoques para resolver el MDVRP. Esta es una metaheurística que explora el espacio de soluciones permitiendo movimientos que empeoran la solución actual para escapar de óptimos locales. Para evitar ciclos, mantiene una lista de movimientos prohibidos o tabú durante un número determinado de iteraciones [33].

También otra de las metaheurísticas de punto único que se destaca es la Búsqueda Local Iterada. Este algoritmo parte de una solución inicial, aplica una perturbación para diversificar la

búsqueda, luego una búsqueda local para alcanzar un nuevo óptimo y finalmente decide si acepta la nueva solución según algún criterio [26].

Entre los poblacionales se destacan bastante los genéticos por la flexibilidad ante ciertas restricciones. Estos algoritmos mantienen una población de soluciones que evoluciona mediante operadores de selección, cruce y mutación. Para el MDVRP, las soluciones se representan como secuencias de clientes. El cruce combina sub-rutas de dos padres para generar un descendiente, reparando la factibilidad si es necesario [33].

Otro de los algoritmos poblacionales que se destacan fue el Algoritmo Colonia de Hormigas. Esta simula el comportamiento de hormigas reales que depositan feromona en el camino. Las hormigas artificiales construyen soluciones probabilísticamente guiadas por la feromona (que refleja la calidad de soluciones previas) y por información heurística local. Para MDVRP, se optimizan rutas por depósito y se añaden operadores de cruce para mejorar el rendimiento [26].

1.3 Algoritmos de agrupamiento para la fase de asignación de clientes a depósitos

La minería de datos es un campo interdisciplinario de la computación que se centra en el descubrimiento de patrones significativos en grandes volúmenes de datos mediante el uso de métodos avanzados de inteligencia artificial, aprendizaje automático, estadística y bases de datos. Su objetivo principal es extraer conocimiento valioso de los datos y transformarlo en una estructura comprensible, ya sea para su análisis o para su almacenamiento en bases de datos con miras a usos posteriores [39].

La minería de datos se basa en una variedad de técnicas y métodos que permiten extraer información útil de grandes volúmenes de datos. Algunas de las técnicas más comunes incluyen [40]:

- Clasificación: es una técnica utilizada para asignar elementos de un conjunto de datos a categorías predefinidas. Esto se logra mediante algoritmos de aprendizaje supervisado, como los árboles de decisión, las máquinas de vectores de soporte (SVM) y las redes neuronales.

- Agrupamiento: es una técnica no supervisada que organiza los datos en clústeres basados en la similitud entre los datos. Esta no requiere categorías predefinidas.

- Regresión: es una técnica utilizada para predecir un valor numérico basado en la relación entre variables.

- Reglas de asociación: identifican relaciones entre variables en un conjunto de datos.

La fase de asignación de clientes a depósitos en MDVRP puede considerarse un problema de agrupamiento, ya que el resultado es un conjunto de clientes agrupados por depósito. Esta interpretación sugiere un enfoque que agrupa a los clientes y luego programa los vehículos en cada grupo. En la literatura el MDVRP puede resolverse en dos etapas: primero, los clientes deben asignarse a los depósitos; luego, se deben construir rutas que vinculen a los clientes asignados al mismo depósito. Un enfoque razonable sería dividir el problema en tantos subproblemas como depósitos existan y resolver cada subproblema por separado, lo cual transforma el MDVRP en múltiples VRP independientes [41]. Por tanto, resulta interesante adaptar los algoritmos de agrupamiento para la fase de asignación en el MDVRP.

Los algoritmos de agrupamiento se basan en la cuantificación de qué tan similares o no sean los objetos del conjunto de datos. Las métricas de distancia desempeñan un papel central en la agrupación de estos objetos en grupos. Entre las medidas comunes de similitud y distancia se incluyen varios tipos de distancia como *Euclidian*, *Manhattan*, *Haversine* y *Chebyshev*. La elección de la medida de proximidad influye en los resultados de la agrupación, ya que los distintos conjuntos de datos requieren técnicas de separación específicas y métricas de similitud adaptadas a sus características. Estos algoritmos de agrupamiento según [42], se pueden clasificar como:

- Agrupamiento particional: en estos algoritmos cada partición representa un grupo y cada objeto debe pertenecer exactamente a un grupo. Si se aplica la partición difusa, permite que los objetos pertenezcan a múltiples grupos con probabilidades. Aquí se encuentran algoritmos como k-means, k-medoids, k-modes y c-means difusos. Estos métodos son sensibles a la inicialización y a los valores atípicos, y se obtienen resultados confiables cuando los grupos son esféricos, de tamaño y distribución uniformes. En la Figura 1.3 se puede apreciar gráficamente el funcionamiento de este tipo de agrupamientos.

- Agrupamiento jerárquico: se dividen en enfoques basados en aglomeración (de abajo hacia arriba) y divisivos (de arriba hacia abajo), que producen una jerarquía de grupos representados como dendrogramas. Los métodos basados en aglomeración comienzan con cada punto de datos como un grupo separado y fusionan los grupos más cercanos de forma iterativa, mientras que los métodos divisivos comienzan con todos los puntos de datos en un grupo y los dividen de forma recursiva. Los criterios de vinculación incluyen la vinculación simple, completa, promedio y el método de Ward, cada uno de los cuales define la proximidad del grupo de manera diferente. Los ejemplos de algoritmos de agrupamiento jerárquico incluyen BIRCH, CURE y ROCK. En la Figura 1.4 se puede apreciar gráficamente el funcionamiento de este tipo de agrupamientos.

- Agrupamiento basado en densidad: los algoritmos de agrupamiento basados en densidad

identifican los agrupamientos como regiones de alta densidad separadas por áreas de baja densidad, sin requerir el número de agrupamientos como entrada. Dentro de esta clasificación se encuentra el agrupamiento espacial basado en densidad de aplicaciones con ruido (DBSCAN), el agrupamiento de puntos para identificar la estructura del agrupamiento (OPTICS) y el agrupamiento de picos de densidad (DPC). Estos métodos pueden presentar dificultades con densidades variables. En la Figura 1.5 se puede apreciar gráficamente el funcionamiento de este tipo de agrupamientos.

[Insertar aquí la figura: Figuras/fig03_particional.png]

Figura 1.3: Representación gráfica de algoritmos de agrupamiento particionales [43].

[Insertar aquí la figura: Figuras/fig04_jerarquico.png]

Figura 1.4: Representación gráfica de algoritmos de agrupamiento jerárquicos [43].

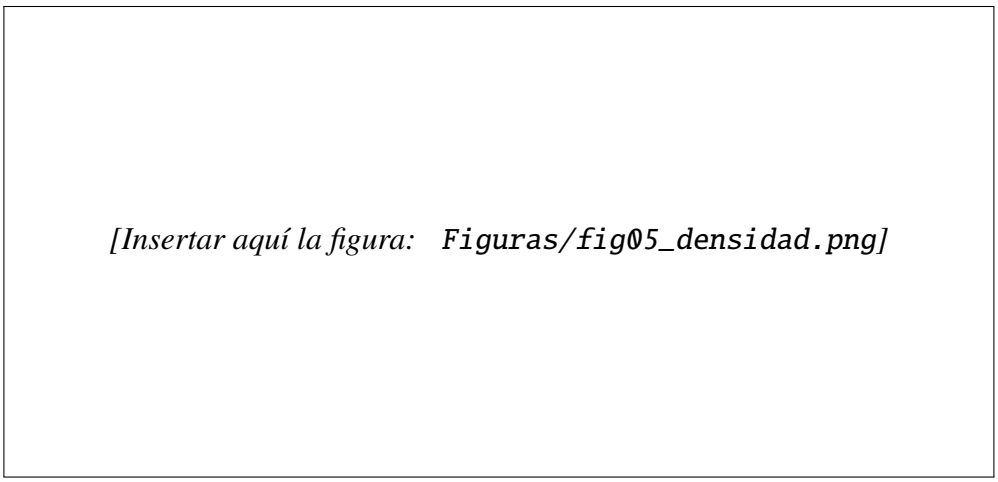


Figura 1.5: Representación gráfica de algoritmos de agrupamiento basados en densidad [43].

Los algoritmos de agrupamiento se aplican al MDVRP principalmente bajo el enfoque "agrupar primero, rutear después", donde primero se agrupan clientes y luego se construyen rutas para cada grupo. Un buen ejemplo de la aplicación de esto es una distribución de alimentos a supermercados en Reino Unido donde se utilizó K-means para agrupar 120 supermercados en cuatro clústeres correspondientes a los depósitos disponibles, minimizando la distancia intra-cluster. Luego se optimizaron rutas independientes para cada clúster. El ahorro en costos operativos fue del 15 % [43].

1.4 Biblioteca de Heurísticas de Asignación en la Planificación de Rutas de Vehículos con Múltiples Depósitos

El área de optimización ha llevado a la creación de diversas bibliotecas en diferentes lenguajes de programación, cada una con su propia variedad de algoritmos y características. A continuación, se presenta una lista no exhaustiva de bibliotecas destacadas en resolver VRP y sus variantes, entre las cuales se incluye el MDVRP. De cada una se ofrece una breve descripción de sus características y usos [15]:

- JSprit: es una biblioteca de código abierto escrita en Java. Es un componente ligero y flexible que resulta fácil de utilizar, fundamentado en una única metaheurística multipropósito que actualmente aborda una variedad amplia de problemas, como el Problema del Agente Viajero y sus variantes de VRP. Aunque no está específicamente diseñada para abordar la fase de asignación en problemas con múltiples depósitos, JSprit puede implementar estrategias de agrupamiento geográfico como un paso preliminar, utilizando técnicas como ordenamientos angulares o agrupaciones espaciales similares al enfoque del clásico

algoritmo de barrido. Estas estrategias permiten organizar a los clientes en función de su proximidad antes de aplicar su metaheurística interna [44].

- *Google Operation Research Tools* (OR-Tools): es un software de código abierto desarrollado en C++, con versiones estables disponibles en Java, C# y Python. Originalmente diseñado para abordar diversos problemas de optimización como la programación lineal, la programación entera y la programación con restricciones, esta herramienta ha ido incorporando gradualmente capacidades para resolver variantes del VRP. Aunque brinda herramientas para construir modelos completos de VRP, su arquitectura flexible permite personalizar fases específicas, como la asignación, mediante programación y heurísticas personalizadas. Además, se pueden integrar algoritmos de agrupamiento para realizar una asignación previa de clientes a los depósitos. También es posible implementar restricciones a medida en el modelo, como límites de capacidad o balanceo de carga, para ajustar la asignación inicial antes de continuar con la planificación de las rutas de los vehículos. Esta herramienta resuelve varias variantes de VRP de manera nativa, pero no el MDVRP [45].

- *Vehicle Routing Open-Source Optimization Machine* (VROOM): es un componente de software para problemas de planificación de rutas de vehículos desarrollado en el lenguaje de programación C++. Esta herramienta utiliza *Open Street Map* (OSM) y *Open Source Routing Machine* (OSRM) como *backend* para obtener rutas y devolver soluciones para las diferentes variantes del VRP. VROOM puede resolver distintas variantes del VRP que incluyen múltiples depósitos, capacidades, ventanas de tiempo, así como entrega y recogida [45].

- Scikit-learn: es una biblioteca de aprendizaje automático en Python que, aunque no está diseñada específicamente para resolver problemas de optimización como el MDVRP, proporciona una amplia gama de algoritmos de agrupamiento y análisis de datos que pueden ser utilizados para abordar la fase de asignación en este tipo de problemas. Algoritmos como K-means, DBSCAN y el agrupamiento jerárquico son útiles para preprocesar los datos y asignar clientes a depósitos de manera eficiente, basándose en proximidad y restricciones de capacidad. Su flexibilidad y compatibilidad con otras bibliotecas de Python la convierten en una herramienta versátil para integrar técnicas de agrupamiento en soluciones personalizadas para el MDVRP [46].

En el ámbito nacional, se han desarrollado diversas soluciones en el contexto académico orientadas a la resolución de problemas de planificación de rutas de vehículos, como parte del proyecto de Optimización y Metaheurísticas de la Facultad de Ingeniería Informática de la Universidad Tecnológica de La Habana “José Antonio Echeverría” (CUJAE). Algunas de estas

herramientas se caracterizan por su enfoque modular y por estar diseñadas para abordar distintas variantes del VRP, con especial énfasis en la utilización de heurísticas y metaheurísticas aplicadas al MDVRP [15]:

- Bibliotecas de Heurísticas de Construcción para Problemas de Planificación de Rutas de Vehículos (BHCVRP): es un componente que implementa diez heurísticas de construcción para resolver siete variantes del VRP, incluyendo el MDVRP. Actualmente las soluciones obtenidas mediante estas heurísticas pueden ser utilizadas como solución final o como parte de una solución más compleja en un contexto donde se empleen metaheurísticas [47].

- Capa de servicio para soluciones a Problemas de Planificación de Rutas de Vehículos (CaSVRP): es una capa de servicios web dirigidos a la resolución de diferentes variantes de VRP mediante algoritmos metaheurísticos. Entre las variantes del VRP que resuelve se halla MDVRP [48]. Esta solución se compone de un módulo de optimización donde se utiliza la Biblioteca de Clases para la Integración de Algoritmos Metaheurísticos (BiCIAM) [49], un módulo para la generación de soluciones iniciales empleando BHCVRP [50] y otro para la generación de nuevas soluciones a partir de otras soluciones haciendo uso de LMOVRP [51].

- Biblioteca de Heurísticas de Asignación para Problemas de Planificación de Rutas de Vehículos con Múltiples Depósitos (BHAVRP): es una biblioteca que resuelve la fase de asignación de clientes a depósitos en el MDVRP. Esta herramienta contiene 18 algoritmos de agrupamiento y heurísticas clásicas de asignación, y es utilizada por otras herramientas como BHCVRP, CaSVRP y TransO [15].

La Tabla 1.1 presenta una comparación entre las bibliotecas antes mencionadas que resuelven la fase de asignación del MDVRP. Es importante resaltar criterios como el hecho de ser de código abierto ya que eso sería de utilidad para seguir mejorando el componente.

Tabla 1.1: Comparación entre las herramientas que resuelven el MDVRP.

Biblioteca	Implementan heurísticas clásicas	Implementan algoritmos de agrupamiento	Lenguajes soportados	Código abierto
JSprit	No	No	Java	Sí
OR-Tools	Sí	No	C++, Python, Java, C#	Sí
VROOM	No	No	C++	No
Scikit-learn	No	Sí	Python	Sí
CaSVRP	Sí	No	Java	Sí
BHCVRP	Sí	No	Java, Python	Sí
BHAVRP	Sí	Sí	Java, Python	Sí

A partir del análisis comparativo de las bibliotecas podemos ver que solo BHAVRP y Scikit-learn ofrecen algoritmos de agrupamiento, lo que los posiciona como herramientas especialmente útiles cuando las necesidades incluyen esta capacidad para el preprocesamiento o modelado del problema. BHAVRP es la única biblioteca específicamente enfocada en resolver la fase de asignación del MDVRP, mediante la implementación de heurísticas clásicas y algoritmos de agrupamiento. Las demás bibliotecas analizadas abordan el problema completo o incluyen la asignación como un paso preliminar, sin especialización explícita en esta etapa.

BHAVRP actualmente ha sido integrado como parte funcional en herramientas como: CaSVRP y BHCVRP. Aunque su uso se ha concentrado en estos sistemas, su arquitectura modular y sus implementaciones en Java y Python permiten su potencial integración en otras aplicaciones compatibles con estos entornos [15].

Como se puede apreciar en la Figura 1.6, el flujo de ejecución comienza en el software que consume BHAVRP, donde se introducen los datos correspondientes a una instancia del problema MDVRP. Estos datos se envían al componente BHAVRP, el cual procede a cargar los datos del problema y a determinar el tipo de distancia que se utilizará en el cálculo de costos. A continuación, BHAVRP verifica si los datos fueron cargados correctamente. Si los datos presentan errores, el proceso se detiene. En caso contrario, se configuran los parámetros de ejecución necesarios, y se genera la matriz de costo correspondiente según la distancia seleccionada. Una vez generada la matriz de costos, el algoritmo se ejecuta en el contexto de BHAVRP, utilizando los parámetros definidos y la matriz de costos generada previamente. Al finalizar la ejecución del algoritmo, BHAVRP genera una solución, que es devuelta al sistema solicitante. Este último obtiene así los resultados de la asignación de clientes a depósitos, completando el ciclo de interacción [15].

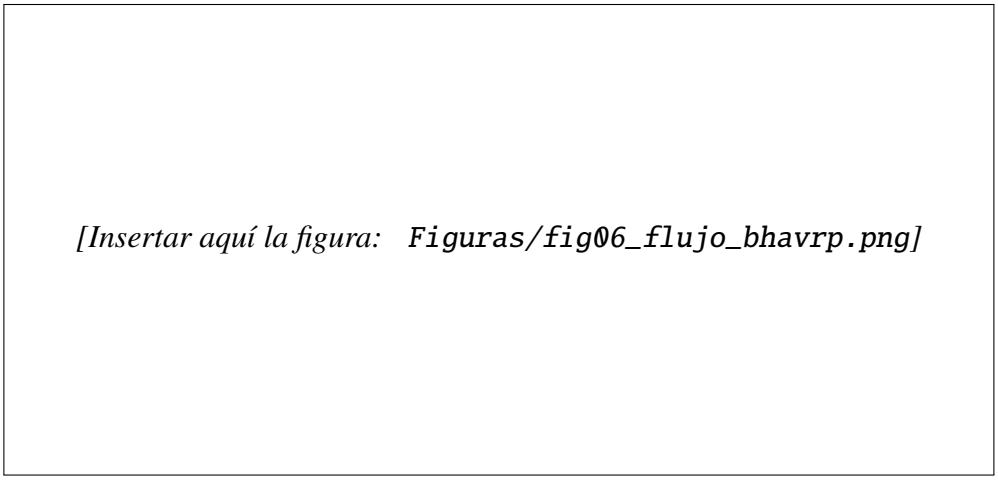


Figura 1.6: Diagrama de actividades del flujo de BHAVRP.

Para soportar la variedad de algoritmos y garantizar su correcta ejecución, BHAVRP presenta una arquitectura n-capas con un enfoque basado en reutilización, promoviendo consistencia con los lenguajes de desarrollo seleccionados [15]. Esta arquitectura se presenta en la Figura 1.7, que ofrece una vista detallada de BHAVRP.

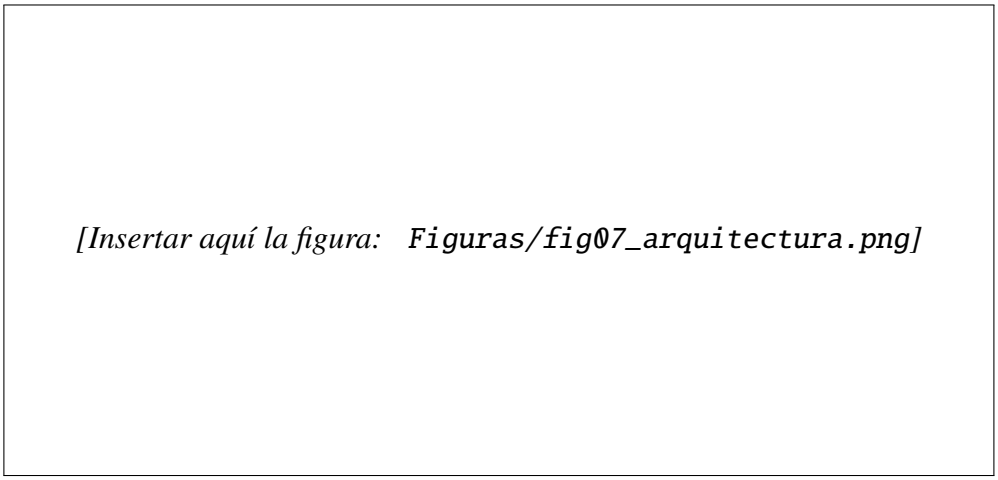


Figura 1.7: Diagrama de arquitectura n-capas con enfoque en reutilización de BHAVRP.

La capa específica contiene los elementos responsables de gestionar la ejecución del sistema, como la clase Controller, que coordina la carga de datos, la configuración de parámetros y el flujo general de ejecución. Por su parte, en la capa general, que representa el núcleo de la biblioteca es responsable de la definición del problema, la modelación de las instancias y la ejecución de los algoritmos de asignación. Por su parte, la biblioteca trabaja con una estructura de datos que representan los elementos clave del MDVRP: clientes, depósitos, y sus respectivas restricciones y parámetros. La estructura básica para utilizar la biblioteca incluye un

encabezado con el número de depósitos, clientes y vehículos disponibles por depósito, seguido de las capacidades máximas de los vehículos. Los clientes y los depósitos están definidos por un identificador y sus coordenadas (x, y). Además, cada cliente tiene una demanda asociada, mientras que cada depósito cuenta con una capacidad equivalente a la suma de las capacidades de los vehículos en su flota. BHAVRP en la capa intermedia contiene varias herramientas. Una de estas permite trabajar tanto con distancias reales como aproximadas (Euclidean, Manhattan, Chebyshev y Haversine). Luego para abordar el cálculo aproximado de las distancias, integra la herramienta OSRM (Open Source Routing Machine), que permite calcular distancias y tiempos de viaje basados en mapas reales, tanto en modalidad local como a través de solicitudes a un servidor en línea. En la capa de software se implementa la comunicación de la herramienta OSRM que expone sus servicios a través de una API HTTP RESTful. El cliente Java (como osrm-java-client) construye las peticiones HTTP y convierte las respuestas a JSON [15].

1.5 Conclusiones parciales

A partir del estudio realizado sobre el marco teórico de los problemas de planificación de rutas de vehículos, específicamente la variante con múltiples depósitos, así como los métodos de solución y algoritmos de Minería de Datos existentes para resolver estos problemas se puede concluir que:

- Los VRP son problemas de optimización combinatoria clasificados como NP-Duro por lo que se utilizan principalmente algoritmos heurísticos y metaheurísticos para su solución. El MDVRP es una de las numerosas variantes del VRP y requiere otro tipo de estrategias de solución.
- Una de las estrategias para la solución del MDVRP es dividir en la fase de asignación y la fase de planificación de las rutas. Además, para esta primera fase de asignación resulta efectivo el uso de algoritmos de minería de datos.
- BHAVRP es una biblioteca que permite resolver esa primera fase de asignación y tiene la facilidad para añadir nuevas heurísticas y algoritmos de agrupamiento gracias a su arquitectura n-capas orientada a reutilización.
- La incorporación de un algoritmo basado en densidad resulta interesante ya que presenta un enfoque diferente a los algoritmos particionales y jerárquicos que ya presenta BHAVRP.
- La incorporación de otro algoritmo de agrupamiento jerárquico a la biblioteca enriquece la biblioteca con otro algoritmo de esta categoría y verifica si UPGMC presenta los mejores resultados gracias a su comportamiento jerárquico o a su naturaleza intrínseca.

Capítulo 2: Incorporación de algoritmos a la Biblioteca de Heurísticas de Asignación del Problema de Planificación de Rutas con Múltiples Depósitos

En este capítulo se presenta la incorporación de nuevos algoritmos a BHAVRP. Se muestra una tabla comparativa entre los algoritmos de agrupamiento ya implementados en la biblioteca y los que se proponen. Además, se presenta una descripción detallada de los algoritmos de agrupamiento propuestos, modificaciones en la arquitectura de la biblioteca para la incorporación de estos y la adaptación para el MDVRP mediante diagramas de actividades.

2.1 Algoritmo DBSCAN

El algoritmo *Density-Based Spatial clustering of Applications with Noise*, pertenece a la familia de algoritmos de agrupamiento basado en densidad, empleado frecuentemente cuando los datos contienen ruido. Este tipo de algoritmos se basan en el concepto de densidad de la vecindad de un punto, donde se calcula un número de puntos que estén dentro de un radio llamado Épsilon [52].

Para el funcionamiento del algoritmo se le pasan los siguientes parámetros [53]:

- Épsilon (ε): el radio de la vecindad alrededor de un punto de datos.
- Puntos mínimos (*minPts*): el número mínimo de puntos de datos que deben estar dentro del radio ε de un punto dado.

Además, DBSCAN clasifica los puntos de los datos de la forma siguiente [52]:

1. Punto núcleo: un punto P es un punto núcleo si al menos *minPts* están a una distancia ε de él o, esos puntos son directamente alcanzables desde P. No es posible tener puntos directamente alcanzables desde un punto que no sea un núcleo.
2. Frontera: un punto Q es frontera si está dentro del radio ε de un punto núcleo P, pero no tiene suficientes puntos vecinos dentro de su propio radio ε para ser considerado núcleo. Estos puntos pertenecen al clúster asociado a P, pero no pueden expandir el grupo hacia nuevos puntos.
3. Ruido: un punto que no sea alcanzable desde cualquier otro punto.

Si P es un punto núcleo, entonces forma un grupo junto a otros puntos (núcleo o no) que sean alcanzables desde él. Cada grupo contiene al menos un punto núcleo. Los puntos no núcleo alcanzables pueden pertenecer al grupo, pero actúan como frontera puesto que no es posible alcanzar más puntos desde estos. Por lo tanto, el tiempo de respuesta se ve comprometido cuando se aplica en el contexto de grandes volúmenes de datos [52].

En la Figura 2.1 se puede apreciar de forma gráfica cómo funciona el algoritmo.

[Insertar aquí la figura: `Figuras/fig08_dbscan.png`*]*

Figura 2.1: Ejemplo de la agrupación de puntos por densidad de DBSCAN.

Para encontrar un clúster, DBSCAN parte de un punto arbitrario cualquiera y recupera todos los puntos alcanzables por su densidad con respecto a su radio ε y los puntos mńimos. Si ese punto es un núcleo, este procedimiento genera un clúster. Si es un punto frontera, DBSCAN se sitúa en el siguiente punto de la base de datos. Dado que se utilizan valores globales para ε y *minPts*, DBSCAN puede fusionar dos clústeres en un solo clúster si dos clústeres de diferente densidad están cerca entre sí. Entonces, dos conjuntos de puntos con al menos la densidad del clúster más delgado solo se separarán si la distancia entre los dos conjuntos es mayor que el radio ε [54].

Si dos clústeres C_1 y C_2 están muy cerca, podría ocurrir que algún punto P pertenezca a ambos. Entonces, P debe ser un punto límite en ambos clústeres; de lo contrario, C_1 sería igual a C_2, dado que se utilizan parámetros globales. En este caso, el punto P se asignará al clúster descubierto primero. Excepto en estas raras situaciones, el resultado de DBSCAN no es independiente del orden en que se visitan los puntos de la base de datos [54].

Con el objetivo de resumir el flujo descrito anteriormente, la Figura 2.2 presenta el diagrama de flujo del algoritmo DBSCAN en su versión más básica. Este diagrama resume las etapas fundamentales del proceso: selección de puntos no visitados, verificación de densidad mediante los parámetros ε y *minPts*, expansión de clústeres y clasificación de puntos como ruido cuando no cumplen el criterio de densidad.

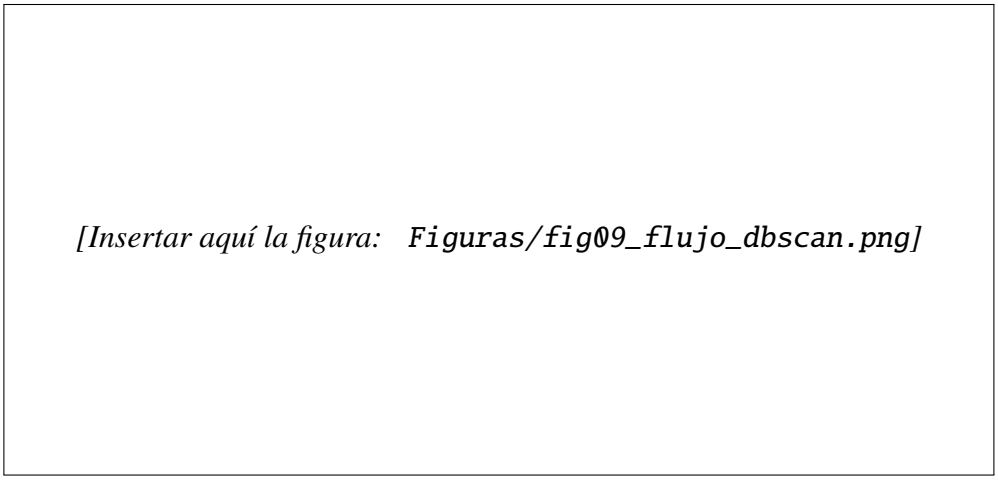


Figura 2.2: Diagrama de flujo del algoritmo DBSCAN.

DBSCAN fue concebido como un algoritmo de agrupamiento basado en densidad espacial, por lo que necesita adaptaciones para resolver MDVRP debido a la presencia de restricciones adicionales como la capacidad de los depósitos y la necesidad de asignación directa hacia depósitos. Por esta razón, se desarrolla una adaptación del algoritmo que incorpora restricciones propias del MDVRP sin perder las funciones originales del enfoque basado en densidad.

La Figura 2.3 muestra el diagrama de actividades correspondiente a la versión adaptada del algoritmo DBSCAN para el MDVRP.

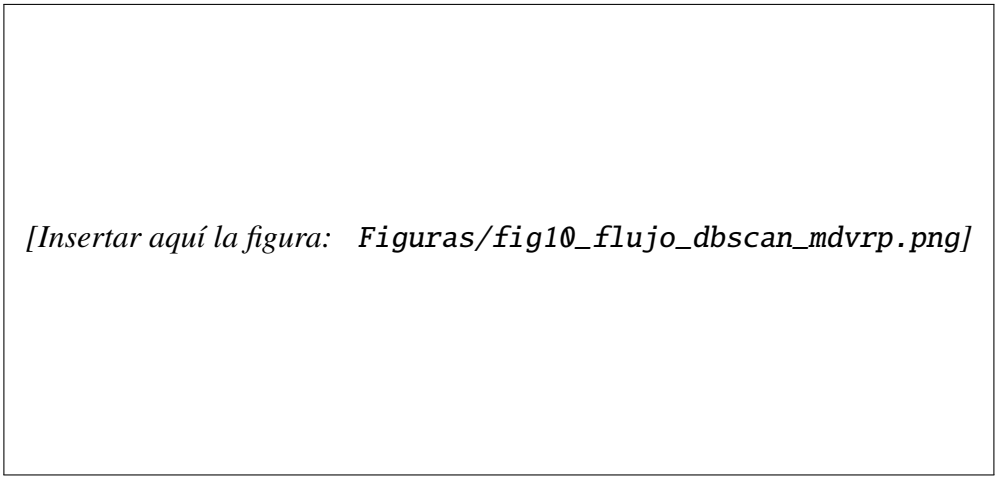


Figura 2.3: Diagrama de flujo del algoritmo DBSCAN a MDVRP.

En esta variante, los clústeres no se crean libremente desde cualquier punto núcleo, sino que se inicializan explícitamente a partir de los depósitos existentes. A partir de cada depósito, el algoritmo expande el clúster considerando los clientes dentro del radio ε, siempre que se cumpla simultáneamente el criterio de densidad (*minPts*) y la restricción de capacidad del depósito.

Además, se incorpora una fase final de reasignación, donde los clientes que no pudieron ser incluidos durante la expansión inicial son evaluados nuevamente y asignados al depósito más cercano con capacidad disponible.

2.2 Algoritmo CURE

En [12] se presenta el algoritmo de agrupamiento jerárquico *Clustering Using Representatives* que usa puntos representativos y un muestreo para gestionar conjuntos de datos más grandes con menores costes computacionales. CURE se puede dividir en dos fases: inicialización y finalización. La inicialización comienza tomando una muestra aleatoria sin reemplazo de la población original y aplicando el agrupamiento jerárquico. Tanto el número de puntos representativos como el factor de contracción se pueden optimizar como parte del ajuste de parámetros. En la etapa de finalización, para cada clúster resultante, selecciona un pequeño conjunto de puntos representativos bien dispersos dentro del clúster y, tras la contracción hacia el centroide del clúster, se asigna el clúster a todos los puntos restantes calculando las distancias hacia todos esos representantes del clúster. Gracias a esto, el algoritmo puede incluso reconocer geometrías de formas arbitrarias, a la vez que es bueno frente a valores atípicos [55].

En la Figura 2.4 se muestra la secuencia de pasos que sigue el algoritmo [12]:

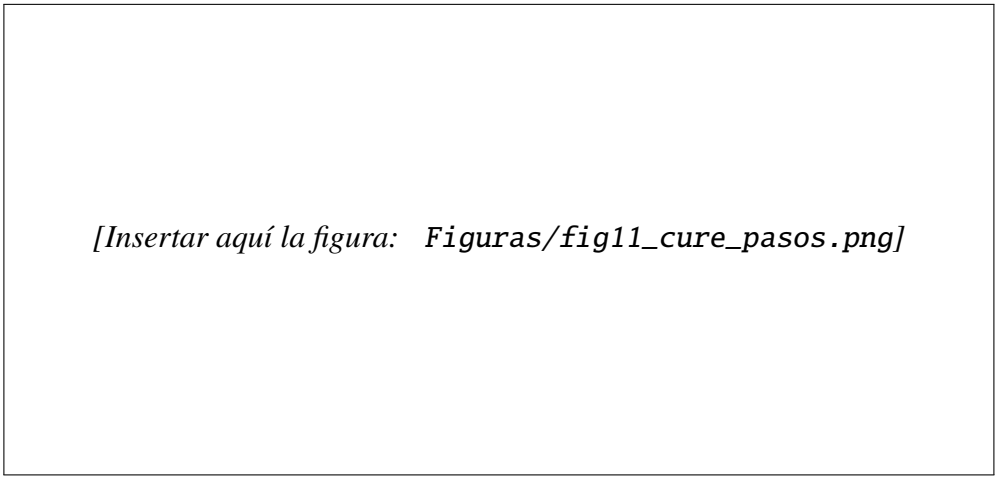


Figura 2.4: Flujo de CURE.

Para comprender el funcionamiento del algoritmo CURE, se mostrará su proceso paso a paso, comenzando con el muestreo del conjunto de datos [56]:

- Muestreo del conjunto de datos: CURE comienza seleccionando una muestra aleatoria representativa del conjunto de datos. El proceso de muestreo reduce el número de puntos de datos, lo que acelera la velocidad de cálculo y mantiene la integridad del clúster. El

muestreo aleatorio simple permite una implementación rápida, pero no logra capturar casos extremos cruciales. Las estrategias de muestreo implementadas por CURE permiten la agrupación posterior para representar con precisión la complejidad y diversidad de todo el conjunto de datos.

- Partición de clústeres: tras muestrear el conjunto de datos, CURE aplica un método jerárquico que divide los datos en subconjuntos manejables. El algoritmo mide las similitudes de los puntos de datos mediante métricas de distancia, algunas de estas son la distancia euclidiana o la distancia de Manhattan. Las métricas de cálculo de distancia son esenciales para determinar la proximidad de los puntos y establecer una base sólida para una fusión de clústeres eficaz. Al inicio del proceso, cada punto de datos funciona como su propio clúster para representar la granularidad fina de los datos. El algoritmo realiza fusiones de clústeres sucesivas utilizando estándares de proximidad para agrupar puntos similares. El algoritmo continúa con las operaciones de fusión de clústeres hasta alcanzar un número definido de clústeres o hasta que se activa una condición de terminación alternativa. El proceso de selección de grupos en CURE crea clústeres que se alinean con las categorías naturales presentes en los datos. Mediante la partición jerárquica, CURE supera las limitaciones de los algoritmos de agrupamiento tradicionales, que requieren que los clústeres tengan formas convexas.

- Selección de puntos representativos: CURE selecciona múltiples puntos representativos para representar cada clúster, en lugar de depender de un único centroide. Estos puntos seleccionados del clúster capturan su rango espacial y estructura. CURE logra un mejor reconocimiento de los límites del clúster y una mejor comprensión de la estructura interna al utilizar múltiples puntos para representar cada clúster. Después de seleccionar los puntos representativos, el algoritmo los mueve hacia la media del clúster con una reducción específica. Esta reducción hace que el algoritmo sea menos reactivo a los valores atípicos, ya que mueve los puntos distantes hacia los puntos centrales del clúster. El éxito del algoritmo depende en gran medida de la cantidad de puntos representativos utilizados durante la ejecución. Usar pocos puntos representativos puede no capturar la complejidad del clúster, mientras que usar demasiados puede aumentar los costos computacionales.

- Fusión de clústeres: tras identificar los puntos representativos, el algoritmo fusiona los clústeres mediante un enfoque iterativo sistemático. El procedimiento se basa en la medición de la distancia entre los puntos representativos de diferentes clústeres. Se utilizan métricas predefinidas, como la distancia euclidiana, para medir las distancias, eliminando así conflictos a la hora de encontrar los clústeres más cercanos. El algoritmo identifica qué pares de clústeres tienen la menor distancia respecto a la separación de los puntos

representativos durante cada paso de la evaluación. El algoritmo toma decisiones precisas sobre la fusión de clústeres, preservando las relaciones espaciales y los patrones naturales de alineación de datos dentro de los clústeres. El proceso itera hasta que se alcanzan algunos clústeres predeterminados o se alcanza otro criterio de terminación. El criterio de terminación depende de factores como la distancia mínima entre clústeres o la similitud máxima permitida dentro de ellos. Esto garantiza que los clústeres generados correspondan a las agrupaciones naturales de los datos y sean lo suficientemente flexibles como para capturar formas irregulares y complejas.

- Manejo de valores atípicos: cuando existen valores atípicos, los resultados se distorsionan porque causan formas incorrectas de los clústeres y malinterpretan la estructura de los datos. El algoritmo CURE resuelve estas limitaciones utilizando múltiples puntos representativos, que identifican con precisión la forma y distribución reales de los clústeres.

La complejidad computacional en el peor caso de este algoritmo de agrupamiento se puede demostrar como $O(n^2 \log(n))$. [12] demuestra que cuando la dimensionalidad de los puntos de datos es pequeña, la complejidad temporal se reduce aún más a $O(n^2)$. Dado que se requiere un espacio lineal, se deduce que la complejidad espacial de este algoritmo es $O(n)$.

Para sintetizar todo el funcionamiento descrito anteriormente, la Figura 2.5 presenta el diagrama de actividades del algoritmo CURE en su versión base. Este diagrama resume el proceso jerárquico de fusión de clústeres, comenzando con cada punto como un clúster independiente y continuando con la unión iterativa de los clústeres más cercanos hasta alcanzar el número deseado k. Destacándose la selección de múltiples puntos representativos por clúster y su posterior contracción hacia el centroide, elemento diferenciador de CURE frente a otros algoritmos jerárquicos tradicionales.

[Insertar aquí la figura: `Figuras/fig12_flujo_cure.png`*]*

Figura 2.5: Diagrama de flujo del algoritmo CURE.

La aplicación directa de CURE al MDVRP no contempla restricciones fundamentales como la capacidad de los depósitos ni la necesidad de que los clústeres estén asociados explícitamente a estos. Por ello, se desarrolló una adaptación del algoritmo que preserva la lógica de representantes múltiples y fusión basada en proximidad, pero incorpora las restricciones pertinentes. La Figura 2.6 muestra el diagrama de actividades correspondiente a la versión adaptada de CURE para el MDVRP.

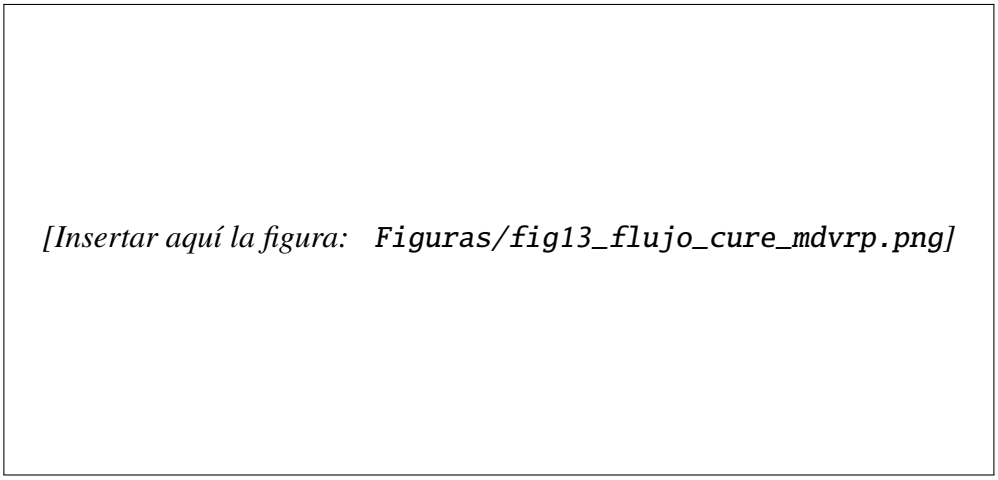


Figura 2.6: Diagrama de flujo del algoritmo CURE adaptado al MDVRP.

En esta variante, los clústeres iniciales se dividen en dos conjuntos diferenciados: clústeres de clientes individuales y clústeres asociados a depósitos.

El proceso iterativo ya no busca reducir el número total de clústeres hasta alcanzar un valor k, sino que se orienta a fusionar progresivamente clientes con clústeres de depósitos, siempre que:

- La distancia mínima entre puntos representativos sea la menor disponible.
- La capacidad del depósito permita absorber la demanda del cliente o subclúster.

Tras cada fusión, se actualizan los puntos representativos del clúster resultante, manteniendo la parte más importante y que caracteriza al algoritmo CURE original.

2.3 Comparación de los algoritmos de agrupamiento

BHAVRP implementa diferentes tipos de algoritmos adaptados para resolver específicamente la fase de asignación del MDVRP desde diferentes enfoques. Estos según la arquitectura de la biblioteca se dividen en heurísticos, de agrupamiento y una categoría otros para los que no se corresponde con alguna de las anteriores. Esta investigación se centra en los algoritmos de agrupamiento. A continuación, se describen los presentes en la biblioteca [15]:

- *K-Means*: asigna seleccionando los clientes con la menor distancia a un depósito.
- *Partitional Around Medoids* (PAM): realiza diferentes asignaciones de los clientes a los depósitos con el objetivo de encontrar la asignación que minimice el costo.
- *Clustering Large Application* (CLARA): es una extensión de PAM para grandes conjuntos de datos, seleccionando muestras representativas.
- *Farthest-First*: asigna seleccionando como centroides los elementos con la mayor distancia entre ellos.
- *Unweighted Pair Group Method with Centroid* (UPGMC): agrupa los clústeres más cercanos hasta que la cantidad de clústeres sea equivalente al número de depósitos.

En la versión actual de BHAVRP, los algoritmos de agrupamiento implementados presentan limitaciones importantes que justifican la incorporación de nuevos algoritmos. En la Tabla 2.1 se muestra una comparación entre todos los algoritmos de agrupamiento de la biblioteca y los algoritmos que se proponen para esta.

Tabla 2.1: Comparación de algoritmos de BHAVRP con los propuestos.

Característica	**KMEANS**	**PAM**	**CLARA**	**Farthest First**	**UPGMC**	**DBSCAN**	**CURE**
Requiere especificar número de clústeres	X	X	X	X	X		X
Detecta formas arbitrarias en la forma de los clústeres						X	X
Escalable a miles de puntos	X		X	X		X	X
Buena identificación de valores atípicos		X	X			X	X

Como se puede apreciar, los algoritmos implementados cuentan con ciertas limitaciones, principalmente ante la forma de los clústeres. Los algoritmos DBSCAN y CURE cubren estas características ausentes en los métodos actuales. Además, a diferencia de UPGMC estos pueden detectar formas arbitrarias en vez de formas circulares e identificar bien los valores atípicos lo cual mejoraría la calidad de asignación con respecto a este. Implementar una adaptación de estos algoritmos para la fase de asignación de clientes a depósitos en MDVRP logrará identificar formas más complejas de clústeres que se adaptan mejor a la realidad geográfica.

2.4 Incorporación de los algoritmos propuestos a BHAVRP

La incorporación de los nuevos algoritmos de agrupamiento a la biblioteca BHAVRP requirió la definición de las siguientes clases: AbstractDensity, DBSCAN, CURE y CureCluster.

Estas clases se integran a la arquitectura existente respetando el patrón de diseño que organiza los algoritmos de agrupamiento dentro de la biblioteca, basado en el patrón *Template Method*. En la Tabla 2.2 se puede apreciar las clases con una pequeña descripción de estas:

Tabla 2.2: Clases implementadas para la incorporación de los nuevos algoritmos de agrupamiento.

Clase	Descripción
AbstractDensity	Clase abstracta que concentra operaciones comunes para algoritmos basados en densidad, como el cálculo de distancias, la búsqueda de vecinos dentro de un radio ε y la asignación de clientes a clústeres.
DBSCAN	Implementación del algoritmo DBSCAN adaptado al MDVRP. Realiza la expansión de clústeres desde los depósitos considerando densidad de clientes, límites de capacidad y posterior reasignación de clientes no asignados.
CURE	Implementación del algoritmo jerárquico CURE adaptado al MDVRP. Ejecuta la asignación de clientes a depósitos mediante la fusión progresiva de clústeres basada en la distancia entre puntos representativos.
CureCluster	Estructura utilizada por CURE para representar un clúster durante el proceso de agrupamiento. Mantiene los clientes del clúster, su demanda acumulada, el centroide y los puntos representativos utilizados para calcular distancias entre clústeres.

Para esta actualización de la biblioteca fue necesario modificar el paquete *Clustering* de la capa general, ya que todas estas clases fueron implementadas en este. Como se puede apreciar en la Figura 2.7, en azul se muestra el paquete modificado:

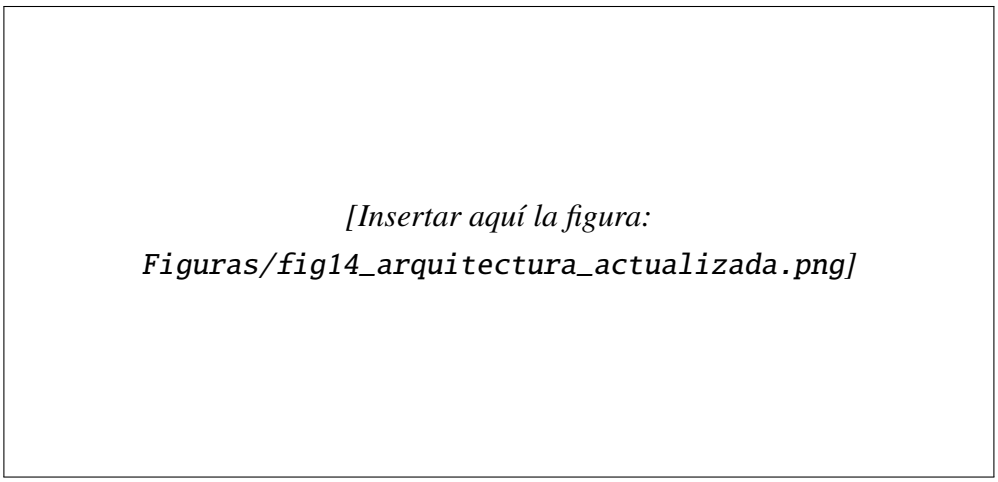


Figura 2.7: Diagrama de arquitectura actualizado.

En BHAVRP, todos los algoritmos de asignación de clientes a depósitos siguen una estructura común que consta de tres fases: inicialización, asignación iterativa y finalización. Independientemente de la heurística específica, el flujo general es siempre el mismo: se preparan las estructuras de datos, se asignan los clientes a los depósitos respetando las restricciones del problema, y se construye la solución final. Sin embargo, los detalles de cómo se realiza la inicialización, cómo se asignan los clientes en cada iteración y cómo se determina la convergencia varían de un algoritmo a otro.

En este contexto, el patrón *Template Method* se utiliza para definir el esqueleto del algoritmo en un método plantilla que establece la secuencia invariante de ejecución. En el diseño de BHAVRP, la clase abstracta AssignmentTemplate (que hereda de la clase base Heuristic) encapsula el flujo del algoritmo en su método plantilla *toClustering* (declarado en la interfaz IAssignment). Este método define la secuencia de pasos invariantes:

1. Inicialización (**initialize**): crea las estructuras necesarias (clústeres, listas de clientes, centroides, etc.) y prepara el estado inicial del algoritmo.

2. Asignación (**assign**): ejecuta el ciclo iterativo donde los clientes se asignan a los clústeres utilizando la matriz de costos, se actualizan los representantes de los clústeres y se verifica si se requiere una nueva iteración.

3. Finalización (**finish**): ensambla la solución final, clasificando los clientes no asignados y los clústeres resultantes.

DBSCAN y CURE respetan este patrón al implementar las operaciones abstractas definidas en la plantilla sin modificar la estructura general del flujo. En ambos casos, el método *to_clustering* mantiene la secuencia establecida por la clase base, mientras que cada algoritmo

redefine los métodos *initialize*, *assign* y *finish* para incorporar su lógica particular de agrupamiento. De esta manera, DBSCAN implementa la expansión de clústeres basada en densidad y CURE el proceso jerárquico de fusión mediante puntos representativos, pero ambos siguen el mismo esquema de ejecución definido por la biblioteca.

La incorporación de estas clases respeta varios principios fundamentales de diseño de software:

- Principio Abierto/Cerrado: la arquitectura base de BHAVRP no requiere modificaciones para incorporar nuevos algoritmos de agrupamiento. La extensión del sistema se logra mediante la creación de nuevas clases que implementan los métodos definidos en la jerarquía existente.

- Principio de responsabilidad única: cada clase mantiene una responsabilidad bien definida dentro del diseño. La estructura general del algoritmo se encuentra encapsulada en la clase base, mientras que las clases concretas se encargan exclusivamente de implementar la lógica particular de cada técnica de agrupamiento.

- Principio de favorecer composición sobre la herencia: en la clase CureCluster en lugar de extender directamente la clase Cluster mediante herencia, CureCluster encapsula una instancia de esta clase y añade información adicional necesaria para el funcionamiento del algoritmo CURE, como los puntos representativos, el centroide y la colección de clientes asociados al clúster.

- Alta cohesión: las operaciones relacionadas con el cálculo de distancias, vecindades y otras funcionalidades compartidas se agrupan en clases específicas como AbstractDensity. Esto permite que cada componente del sistema concentre funcionalidades relacionadas entre sí.

- Bajo acoplamiento: las implementaciones concretas dependen únicamente de las interfaces y estructuras generales proporcionadas por BHAVRP, lo que facilita la sustitución o incorporación de nuevas heurísticas sin afectar el resto del sistema.

En la Figura 2.8 se puede apreciar la jerarquía de clases donde se aplica el patrón Template Method y la implementación de las nuevas clases respetando este. La clase AssignmentTemplate actúa como clase base con el método plantilla, y las distintas heurísticas de asignación sobrescriben los métodos abstractos según su lógica específica.

[Insertar aquí la figura: `Figuras/fig15_diagrama_clases.png`*]*

Figura 2.8: Diagrama de clases actualizado.

2.5 Conclusiones parciales

A partir del análisis sobre los algoritmos DBSCAN y CURE, la comparación con los algoritmos de agrupamiento de BHAVRP, además del diseño de la adaptación de estos algoritmos a MDVRP se concluye que:

- Los algoritmos DBSCAN y CURE suponen una buena inclusión a BHAVRP gracias a su capacidad de generar clústeres con formas más complejas y la detección de valores atípicos. DBSCAN siendo un buen primer acercamiento a los algoritmos basados en densidad dentro de la biblioteca y CURE una alternativa bastante robusta gracias al uso de puntos representativos.
- DBSCAN, originalmente diseñado para detección de densidad, puede reinterpretarse como un mecanismo de expansión controlada desde depósitos, convirtiéndolos en núcleos de agrupamiento.
- CURE, como algoritmo jerárquico, ofrece mayor flexibilidad al permitir fusiones progresivas basadas en distancia mínima entre puntos representativos.
- Ambas adaptaciones mantienen la esencia estructural de los algoritmos clásicos, pero introducen mecanismos adicionales de validación que garantizan factibilidad logística.

Capítulo 3: Análisis experimental

En este capítulo se presentan las pruebas funcionales y unitarias para las funcionalidades principales de los algoritmos implementados, además de un estudio comparativo entre los resultados obtenidos por los algoritmos de agrupamiento de la biblioteca y los nuevos propuestos usando las métricas de evaluación que contiene BHAVRP. También una prueba estadística no paramétrica entre los dos algoritmos jerárquicos.

Todos los experimentos se llevan a cabo en un entorno controlado, utilizando un ordenador con las siguientes especificaciones: procesador AMD A6-7310 APU with AMD Radeon R4 Graphics de 2.4GHz, 4 GB de memoria RAM y sistema operativo Linux Mint de 64 bits, usando el IDE Intellij IDEA con Java en su versión 23.

3.1 Pruebas funcionales

Con el propósito de garantizar la correcta integración y funcionamiento de los nuevos algoritmos implementados en BHAVRP, se desarrolla un conjunto de pruebas unitarias usando el componente Junit para validar las funciones esenciales de los algoritmos: método *distance* de la clase DBSCAN y constructor, así como método *merge* de la clase CureCluster y el método *calculateClusterDistance* de la clase CURE. Estas pruebas funcionales se realizan previamente al análisis de calidad y rendimiento de los algoritmos de asignación sobre instancias.

3.1.1 Prueba del método *distance* en DBSCAN

El método *distance* de la clase DBSCAN es fundamental para el algoritmo de agrupamiento basado en densidad, ya que calcula la distancia euclidiana entre dos puntos geográficos representados por objetos de la clase *Location*. Para asegurar su correcto funcionamiento se diseñan pruebas unitarias que verifican:

- Cálculo correcto de distancias: se comparan distancias conocidas, como la distancia entre (0,0) y (3,4) que debe ser 5.0 por la métrica euclidiana.
- Distancia a sí mismo: se verifica que la distancia de un punto a sí mismo sea cero, lo cual es una propiedad básica de cualquier métrica.
- Manejo de coordenadas negativas: se prueba con puntos que incluyen coordenadas negativas para asegurar que el cálculo no se ve afectado por el signo.
- Simetría: se comprueba que la distancia entre A y B sea igual a la distancia entre B y A, propiedad necesaria para cualquier función de distancia.

Adicionalmente, se verifica que el método sea consistente para diferentes instancias y que el tipo de distancia por defecto sea la euclidiana. En la Figura 3.1 se puede apreciar el resultado proporcionado por el IDE a esta prueba.

[Insertar aquí la figura: `Figuras/fig16_prueba_distance.png`*]*

Figura 3.1: Resultados de las pruebas del método distance.

3.1.2 Pruebas del constructor y método merge de la clase CureCluster

La clase CureCluster representa un clúster en el algoritmo CURE. Su correcta construcción y la operación de fusión (*merge*) son críticas para el proceso jerárquico. Las pruebas unitarias se centran en:

Creación de clústeres:

- Con un solo cliente: se verifica que el clúster se cree correctamente, que contenga al cliente, que el centroide se calcule adecuadamente y que la demanda total sea la esperada.
- Con múltiples clientes: se comprueba que el centroide sea el promedio de las coordenadas de los clientes y que la demanda sea la suma de las demandas individuales.
- Se verifica que los puntos representativos se generen correctamente tras la creación.

Operación de fusión (merge):

- Se fusionan dos clústeres y se comprueba que el número de clientes resultante sea la suma de ambos, que la demanda total sea la suma y que el centroide se actualice correctamente.
- Se verifica que la lista de identificadores de clientes en el clúster base se actualice adecuadamente.

- Se prueba que la fusión no altere la integridad de los datos y que los puntos representativos del nuevo clúster se calculen apropiadamente.

Casos bordes:

- Manejo de listas vacías de clientes en la creación.
- Fusión de un clúster consigo mismo (aunque no sea una operación válida en la práctica, se verifica que no lance excepciones inesperadas).

Estas pruebas aseguran que la estructura de datos subyacente al algoritmo CURE sean confiables y que las operaciones de agrupamiento se realicen de forma correcta. En la Figura 3.2 se puede apreciar el resultado proporcionado por el IDE a esta prueba.

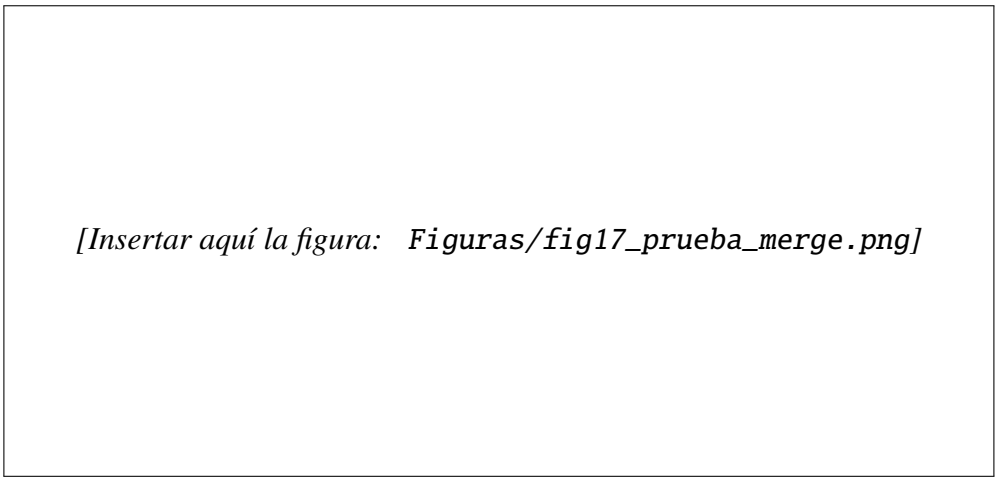


Figura 3.2: Resultados de las pruebas de constructor y del método merge.

3.1.3 Pruebas del método *calculateClusterDistance* de la clase CURE

El método *calculateClusterDistance* es uno de los métodos más importantes del algoritmo CURE, ya que determina la distancia entre dos clústers utilizando los puntos representativos contraídos. Las pruebas unitarias diseñadas verifican:

- Cálculo de distancia entre clústeres separados: se crean clústeres con puntos en disposiciones horizontales y verticales, y se calcula la distancia esperada manualmente a partir de los puntos contraídos. Luego se compara con el resultado del método, asegurando que coincida dentro de una tolerancia.
- Se prueba con clústeres que tienen múltiples puntos, comprobando que la distancia mínima entre los puntos contraídos sea la retornada.

- Simetría: se verifica que la distancia entre el clúster A y el clúster B sea igual a la distancia entre B y A.
- Manejo de clústeres con un solo punto: en este caso, el centroide coincide con el punto, por lo que el punto contraído es el mismo.
- Coordenadas negativas: se prueban clústeres con puntos en coordenadas negativas para garantizar que el cálculo sea correcto en todo el plano.
- Uso de puntos representativos contraídos: se comprueba que los puntos utilizados en el cálculo sean efectivamente los puntos contraídos y no los originales.
- Valores positivos y finitos: se asegura que la distancia calculada sea siempre positiva y finita para clústeres separados.

Estas pruebas garantizan que la métrica de distancia entre clústeres en el algoritmo CURE sea la adecuada para el proceso de agrupamiento jerárquico. En la Figura 3.3 se puede apreciar el resultado proporcionado por el IDE a esta prueba.

[Insertar aquí la figura: `Figuras/fig18_prueba_calcdist.png`*]*

Figura 3.3: Resultados de las pruebas del método calculateClusterDistance.

3.2 Pruebas unitarias

Además de las pruebas funcionales de los métodos esenciales, se realizaron pruebas de caja negra para verificar el comportamiento global de los algoritmos DBSCAN y CURE en escenarios extremos, asegurando que las adaptaciones al MDVRP respetan las restricciones de capacidad y asignación.

En la Tabla 3.1, DBSCAN demuestra que su fase de expansión controlada desde los depósitos, junto con la reasignación final, logra respetar los límites de capacidad incluso cuando

la densidad de clientes supera la capacidad disponible. La Tabla 3.2 muestra que CURE, mediante su mecanismo de fusión jerárquica controlada, asigna los clientes de manera que no se exceda la capacidad de ningún depósito, dejando como no asignados únicamente aquellos que no pueden ser absorbidos por ningún depósito sin violar la restricción. Estos resultados confirman que ambas adaptaciones cumplen con las reglas de negocio del MDVRP.

Tabla 3.1: Pruebas unitarias de caja negra del algoritmo DBSCAN.

No	Descripción	Caso prueba	Juego de Datos	Resultados esperados	Resultados Reales
1	Verificar asignación básica de clientes a depósitos cuando todos se encuentran dentro del radio ε.	Asignación básica DBSCAN	2 depósitos y 6 clientes ubicados cerca de cada depósito. Demandas pequeñas dentro de la capacidad.	Cluster 7 = [1,2,3], Cluster 8 = [4,5,6], no asignados = 0	Cluster 7 = [1,2,3], Cluster 8 = [4,5,6], no asignados = 0
2	Validar que el algoritmo respeta la capacidad máxima del depósito.	Capacidad limitada DBSCAN	1 depósito con capacidad limitada y varios clientes cuya suma de demandas supera la capacidad.	Cluster 6 = [1,2], no asignados = [3,4,5]	Cluster 6 = [1,2], no asignados = [3,4,5]
3	Validar la reasignación final de clientes no asignados.	Reasignación DBSCAN	Clientes alejados del depósito inicial pero cercanos a otro depósito con capacidad disponible.	Cluster 6 = [1,2,3], Cluster 7 = [4,5], no asignados = 0	Cluster 6 = [1,2,3], Cluster 7 = [4,5], no asignados = 0
4	Validar excepción cuando un deposito no puede alcanzar ningún cliente	Depósito Vacío	Pocos clientes muy alejados de los depósitos.	ClusterExcepción	ClusterExcepción
5	Validar excepción cuando los datos son erróneos.	Error en los Datos	Instancia sin depósitos.	ProblemException	ProblemException

Tabla 3.2: Pruebas unitarias de caja negra del algoritmo CURE.

No	Descripción	Caso prueba	Juego de Datos	Resultados esperados	Resultados Reales
1	Verificar la asignación básica de clientes a depósitos mediante fusión de clústeres.	Asignación básica CURE	2 depósitos y varios clientes distribuidos en dos regiones cercanas a cada depósito.	Cluster 7 = [1,2,3], Cluster 8 = [4,5,6], No asignados: 0	Cluster 7 = [1,2,3], Cluster 8 = [4,6,5], No asignados: 0
2	Validar que la fusión de clústeres respeta la capacidad del depósito.	Capacidad CURE	Clientes cercanos a un depósito cuya demanda acumulada excede la capacidad disponible.	Cluster 6: [1,2], No asignados: [3,4,5]	Cluster 6: [1,2], No asignados: [3,4,5]
3	Verificar el comportamiento cuando no es posible fusionar más clústeres.	No asignables CURE	Clientes restantes cuya asignación provocaría exceder la capacidad de todos los depósitos.	Cluster 5 = [1], Cluster 6 = [3], No asignados: [2,4]	Cluster 5 = [1], Cluster 6 = [3], No asignados: [2,4]
4	Validar excepción cuando los datos son erróneos.	Error en los Datos	Instancia sin depósitos.	ProblemException	ProblemException

3.3 Comparación entre los algoritmos de agrupamiento

Con el objetivo de analizar el comportamiento y la calidad de las asignaciones generadas por los algoritmos implementados en BHAVRP, se diseña un experimento enfocado en instancias utilizadas en trabajos anteriores [14], [15]. Este experimento busca comparar el rendimiento de cada algoritmo de asignación en condiciones controladas, evaluando los resultados mediante el conjunto de métricas internas incluidas dentro de la biblioteca. El estudio experimental se lleva a cabo empleando las 33 instancias originalmente introducidas por [57] en su trabajo sobre el MDVRP. Estas instancias tienen amplio reconocimiento en el ámbito científico de la optimización de rutas. La Tabla 3.3 detalla, para cada instancia del problema, el número total de clientes, la cantidad de depósitos, la flota de vehículos asignada a cada depósito y su capacidad. Cabe destacar que aquellas instancias que figuran con el mismo identificador en la columna 'ID' presentan idénticas características en cuanto a número de clientes, depósitos, vehículos por depósito y capacidad de los vehículos.

Para el experimento se usaron solo los algoritmos de agrupamiento de BHAVRP ya mencionados en la sección 2.3. Al ser todos estos algoritmos deterministas, cada algoritmo se ejecuta una sola vez por instancia, utilizando como tipo de distancia aproximada la distancia

euclidiana. Para analizar la calidad de las asignaciones generadas por cada algoritmo, se utilizan todas las métricas de validación interna incorporadas en BHAVRP. Estas métricas permiten evaluar cuantitativamente aspectos clave como la cohesión y la separación entre clústeres. En la Tabla 3.4, se presentan las métricas a utilizar con sus siglas, el criterio que evalúan y una descripción.

Tabla 3.3: Descripción de las instancias.

ID	Total de clientes	Total de depósitos	Total de vehículos por depósito	Capacidad de los vehículos
p01	50	4	4	80
p02	50	4	2	160
p03	75	5	3	140
p04	100	2	8	100
p05	100	2	5	200
p06	100	3	6	100
p07	100	4	4	100
p08	249	2	14	500
p09	249	3	12	500
p10	249	4	8	500
p11	249	5	6	500
p12, p13, p14	80	2	5	60
p15, p16, p17	160	4	5	60
p18, p19, p20	240	6	5	60
p21, p22, p23	360	9	5	60
pr1	48	4	1	200
pr2	96	4	2	195
pr3	144	4	3	190
pr4	192	4	4	185
pr5	240	4	5	180
pr6	288	4	6	175
pr7	72	1	6	200
pr8	144	2	6	190
pr9	216	6	3	180
pr10	288	6	4	170

Tabla 3.4: Resumen de las métricas incorporadas en BHAVRP.

Métrica	Siglas	¿Qué evalúa?	Descripción
Suma de Errores al Cuadrado	SSE	Cohesión	Suma de las distancias cuadráticas entre cada punto y centroide de su grupo. **Minimizar**.
Índice de Ball-Hall	BH	Cohesión	Mide la dispersión promedio dentro de los clúster **Minimizar**.
Suma de Cuadrados entre Clústeres	SSB	Separación	Evalúa la separación entre los grupos considerando distancia entre sus centroides y el centro global de l datos. **Maximizar**.
Índice de Calinski-Harabasz	CH	Cohesión y Separación	Relación entre la dispersión entre clústeres (SSB) y dispersión dentro de los clústeres (SSE). **Maximizar**
Índice de Dunn	DI	Cohesión y Separación	Relación entre la distancia mínima inter-clúster y el d metro máximo intra-clúster (el tamaño del grupo m grande). **Maximizar**.
Índice de Davies-Bouldin	DB	Cohesión y Separación	Analiza la relación entre la dispersión interna de l clústeres y la distancia entre sus centroides. **Minimiza**
Coeficiente de Silueta	CS	Cohesión y Separación	Combina cohesión y separación para cada punto. **M ximizar**.
Índice de Xie-Beni	XB	Cohesión y Separación	Penaliza los agrupamientos donde los puntos están a jados de sus centroides y los que están muy próxim entre sí. **Minimizar**.
Índice de Ray-Turi	RT	Cohesión y Separación	Evalúa la relación entre la dispersión dentro de los clú teres y la distancia mínima entre los centroides de l clústeres. **Minimizar**.

3.3.1 Análisis estadístico

A partir de los experimentos realizados, se decide utilizar la prueba estadística no paramétrica de Friedman N × N [58], la cual posibilita enfrentar por pares todos los algoritmos examinados.

En la Tabla 3.5 se presenta para cada algoritmo su valor promedio por cada métrica en todas las instancias. Las celdas destacadas en verde reflejan las mejores posiciones, mientras que las celdas en rojo indican los desempeños más bajos. Al observar los rankings promedio se distingue una separación en el rendimiento de los algoritmos analizados. Este examen se ve complementado por los análisis post-hoc.

Tabla 3.5: Resultados de la prueba de Friedman NxN.

Algoritmo	SSE	BH	SSB	CH	DI	CS	DB	XB	RT	Tiempo
CURE	2.0303	2.2424	3.0909	2.1515	2.7879	2.2424	2.5152	2.5303	2.5303	2.0455
DBSCAN	4.4697	4.3939	3.9242	4.6818	5.4242	4.6212	4.2879	4.0455	4.0455	1.0455
Farthest-First	3.3333	3.3636	3.3939	3.1818	3.5758	3.1364	3.2424	3.1212	3.1212	4.1970
KMEANS	3.0606	3.1818	3.4242	2.9091	3.5606	2.8788	3.0909	2.9545	2.9545	3.8182
UPGMC	3.7121	3.3030	4.1364	3.9242	1.3030	3.8636	3.7727	4.0455	4.0455	3.9848
PAM	5.3636	5.3636	4.5152	5.1818	5.3636	5.1364	5.0303	5.3333	5.3333	7
CLARA	6.0303	6.1515	5.5152	5.9697	5.9848	6.1212	6.0606	5.9697	5.9697	5.9091

A continuación, se presenta la interpretación de los resultados obtenidos en el ranking:

- CURE obtiene los mejores rankings en las métricas de cohesión y en la única de separación obtuvo el mejor. También obtuvo el mejor ranking en la mayoría de las combinadas, situándose además como el segundo más rápido. Esto indica que el nuevo algoritmo jerárquico es capaz de generar agrupaciones compactas y bien separadas en un buen tiempo.

- UPGMC presenta un comportamiento muy bueno en separación, pero sus rankings en cohesión y métricas combinadas son peores que los de CURE. Esto sugiere que, aunque UPGMC logra clústeres muy separados, la consistencia interna de sus grupos es inferior.

- DBSCAN destaca por ser el algoritmo más rápido. Sin embargo, muestra un desempeño moderado en cohesión y pobre en separación y métricas combinadas.

- K-Means y Farthest-First mantienen posiciones intermedias en todas las categorías, mientras PAM y CLARA ocupan las últimas posiciones, reflejando peor calidad de agrupamiento y además los peores tiempos.

Una vez obtenidos los rankings promedio de Friedman, se aplicaron los procedimientos *post-hoc* de Bergmann [59], Holm [60] y Shaffer [61] para controlar la tasa de error familiar. A continuación, en la Tabla 3.6 se resumen los resultados de los procedimientos *post hoc* de Bergmann, Holm y Shaffer para cada métrica evaluada. Se indican las diferencias estadísticamente significativas ($\alpha = 0.05$) y más relevantes para el análisis. Estos se emplearon porque son métodos estándar para el control de rechazar una hipótesis nula verdadera tras un test de Friedman. Holm se usa por su simplicidad y aceptación general; Shaffer mejora la potencia al explotar las relaciones lógicas entre las hipótesis de comparaciones por pares; y Bergmann proporciona la máxima potencia al determinar conjuntos de hipótesis aceptables.

Tabla 3.6: Resumen análisis post-hoc.

Métrica	Comparación	Holm	Shaffer	Bergmann
SSE	CURE > CLARA, PAM	0	0	0
	CURE > UPGMC	0.020341	0.017212	0.014082
	CURE > DBSCAN	0.000076	0.000067	0.000049
	KMEANS > CLARA	0	0	0
	KMEANS > PAM	0.000223	0.000223	0.000149
	FF > CLARA	0.000007	0.000006	0.000004
DI	UPGMC > CURE	0.036664	0.036664	0.026189
	UPGMC > CLARA, PAM	0	0	0
	CURE > CLARA	0	0	0
	CURE > PAM	0.000020	0.000019	0.000009
CH	CURE > CLARA, PAM	0	0	0
	CURE > UPGMC	0.011155	0.009439	0.007723
	KMEANS > CLARA	0	0	0
	FF > CLARA	0.000003	0.000002	0.000002
CS	CURE > CLARA, PAM	0	0	0
	CURE > UPGMC	0.027603	0.025303	0.020703
	KMEANS > CLARA	0	0	0
DB	CURE, KMEANS > CLARA	0	0	0
	FF > CLARA	0.000002	0.000002	0.000001
XB y RT	CURE, KMEANS > CLARA	0	0	0
	FF > CLARA	0.000002	0.000001	0.000001
Tiempo	DBSCAN > PAM, CLARA, FF	0	0	0
	DBSCAN > UPGMC	0.000001	0	0
	DBSCAN > KMEANS	0.000002	0.000002	0.000001

El análisis *post-hocs* muestra las siguientes conclusiones:

- CURE es el algoritmo con mejor calidad de agrupamiento, superando significativamente a UPGMC en la mayoría de las métricas combinadas y mostrando una clara ventaja en cohesión y tiempo de ejecución.
- UPGMC solo conserva una ventaja significativa en el Índice de Dunn, lo que confirma que su buen comportamiento previo se debe a sus características particulares y no a su naturaleza jerárquica.
- DBSCAN domina en tiempo de ejecución, pero su calidad es moderada y significativamente inferior a CURE en la mayoría de los aspectos.
- KMEANS y Farthest-First ocupan posiciones intermedias, superando a PAM y CLARA.

- PAM y CLARA son consistentemente los peores en calidad y tiempo, siendo superados por todos los demás algoritmos en múltiples métricas.

Es importante destacar que DBSCAN, en su adaptación con restricciones de capacidad, resulta extremadamente rápido, pero produce agrupaciones de calidad moderada en términos de separación. Esto puede ser aceptable en contextos donde la velocidad computacional sea prioritaria y se pueda tolerar una asignación menos refinada. No obstante, para instancias donde se requiere alta calidad en la creación de rutas posterior, otros algoritmos como CURE o K-Means serían más adecuados.

Por último, los rankings muestran que CURE supera a UPGMC en cohesión y en la mayoría de las métricas combinadas, mientras que UPGMC solo le gana en el índice de Dunn. Esta diferencia sugiere que la estrategia de múltiples puntos representativos y contracción de CURE permite una mejor representación de la forma de los clústeres. Esto tiene como resultado grupos más homogéneos internamente sin sacrificar en exceso la separación. No obstante, para confirmar si estas diferencias son estadísticamente significativas, se requiere un análisis específico entre ambos algoritmos, el cual se aborda en el próximo epígrafe mediante la prueba estadística no paramétrica de Wilcoxon.

3.4 Comparación entre los algoritmos de agrupamiento jerárquico

En este experimento se compara directamente UPGMC y CURE utilizando la prueba de Wilcoxon. Esto permitirá determinar si las diferencias observadas en los rankings de Friedman son estadísticamente significativas para cada métrica. La prueba estadística no paramétrica de Wilcoxon se utiliza para dos muestras relacionadas, en este caso para dos algoritmos ejecutados en las mismas instancias [62].

Con el objetivo de fortalecer el análisis experimental, se incorporan 11 instancias adicionales, alcanzando un total de 44 instancias para la evaluación de estos dos algoritmos. Estas instancias provienen de la investigación realizada en [63], y se detallan en la Tabla 3.7.

Tabla 3.7: Descripción de las nuevas instancias.

ID	Total de clientes	Total de depósitos	Total de vehículos por depósito	Capacidad de los vehículos
gj1	50	2	2	492
gj2	75	3	3	415
gj3	100	2	2	841
gj4	150	4	4	651
gj5	199	5	5	758
gj6	75	2	2	669
gj7	125	3	3	768
gj8	249	4	4	1219
gj9	60	2	2	579
gj10	80	3	3	507
gj11	120	5	5	1000

Ambos algoritmos se ejecutaron bajo las mismas condiciones experimentales descritas en la sección 3.3, y los resultados para cada métrica de validación interna fueron sometidos a la prueba de Wilcoxon.

A continuación, en la Tabla 3.8, se presentan e interpretan los resultados de la prueba de Wilcoxon para cada una de las métricas.

Tabla 3.8: Resumen de los resultados de Wilcoxon.

Métrica	p-value (CURE vs UPGMC)	p-value (UPGMC vs CURE)	Interpretación ($\alpha = 0.05$)
CH	0.000000000122	≥ 0.2	CURE es significativamente mejor
DI	≥ 0.2	0.0000616	UPGMC es significativamente mejor
CS	0.0000000164	≥ 0.2	CURE es significativamente mejor
SSB	0.00129	≥ 0.2	CURE es significativamente mejor
BH	≥ 0.2	≥ 0.2	No hay diferencia significativa
DB	0.06164	≥ 0.2	No significativo ($p > 0.05$) aunque hay tendencia
RT	0.000178	≥ 0.2	CURE es significativamente mejor
SSE	0.00000000913	≥ 0.2	CURE es significativamente mejor
Tiempo	0.000000000000114	≥ 0.2	CURE es significativamente mejor
XB	0.000153	≥ 0.2	CURE es significativamente mejor

La Tabla 3.8 ofrece una visión concluyente de la comparación entre ambos algoritmos

jerárquicos. CURE emerge como el algoritmo dominante en este estudio comparativo, superando a UPGMC de manera significativa en siete de las nueve métricas de calidad de agrupamiento y en el tiempo de ejecución.

- CURE demuestra ser superior en las métricas de SSE y SSB. Así como en Calinski-Harabasz (CH), el Coeficiente de Silueta (CS) y el Índice de Xie-Beni (XB). Esto indica que, en promedio, la estructura de agrupamiento creada por CURE es de mayor calidad que la de UPGMC.
- El único aspecto donde UPGMC supera significativamente a CURE es en el Índice de Dunn (DI).
- En el Índice de Ball-Hall (BH), ambos algoritmos se comportan de manera estadísticamente similar. Por otro lado, en el Índice de Davies-Bouldin (DB), la superioridad de CURE no es tan significativa.
- La clara ventaja en términos de tiempo de ejecución es para CURE. No solo produce mejores agrupaciones en la mayoría de los casos, sino que lo hace significativamente más rápido.

La evidencia demuestra que la naturaleza jerárquica por sí sola no garantiza superioridad. El algoritmo UPGMC, muestra el mejor desempeño en la métrica del Índice de Dunn que es separación entre los clústeres más cercanos, es superado de manera estadísticamente significativa por CURE. Además, CURE logra estos mejores resultados con un costo computacional significativamente menor.

3.5 Conclusiones parciales

A partir del estudio experimental realizado en este capítulo, se pueden extraer las siguientes conclusiones:

- Las pruebas unitarias aplicadas a los nuevos algoritmos (DBSCAN y CURE) confirmaron el correcto funcionamiento de sus métodos fundamentales, garantizando su integración fiable en la biblioteca BHAVRP.
- CURE se destaca como una buena alternativa, con los mejores rankings globales en la mayoría de las métricas, mientras que DBSCAN ofrece la opción más rápida, aunque de calidad más limitada.

- El análisis detallado mediante la prueba de Wilcoxon entre los dos algoritmos jerárquicos determina que la buena actuación previa de UPGMC se debía a sus características particulares y no a su naturaleza jerárquica. CURE, que presenta una mecánica basada en puntos representativos, demuestra ser estadísticamente superior en la mayoría de las métricas de calidad y en tiempo de ejecución.

Conclusiones

La presente investigación ha permitido abordar el problema planteado, logrando incorporar nuevos algoritmos de agrupamiento en la Biblioteca de Heurísticas de Asignación para Problemas de Planificación de Rutas de Vehículos con Múltiples Depósitos (BHAVRP). A partir del trabajo realizado, se pueden extraer las siguientes conclusiones generales.

El estudio bibliográfico desarrollado evidencia que la fase de asignación de clientes a depósitos en el MDVRP puede ser abordada eficazmente mediante técnicas de agrupamiento de la minería de datos, aprovechando su capacidad para descubrir estructuras en los datos geográficos. Asimismo, el análisis comparativo de bibliotecas existentes reafirma que BHAVRP constituye una buena alternativa dentro del MDVRP, al ser la única especializada en esta fase y con una arquitectura abierta y extensible que facilita la incorporación de nuevos algoritmos. La implementación de los algoritmos DBSCAN y CURE aumenta la diversidad de enfoques de la biblioteca. La incorporación de restricciones de capacidad y adaptar los mecanismos de expansión y fusión, demuestra que es posible aplicar técnicas clásicas de minería de datos a contextos de optimización logística, garantizando la factibilidad de las soluciones generadas.

Por último, el análisis experimental desarrollado valida el correcto funcionamiento de los algoritmos implementados a través de pruebas unitarias. El primer experimento posiciona a CURE como el algoritmo de rendimiento superior, obteniendo los mejores rankings en la mayoría de las métricas de calidad de agrupamiento y un tiempo de ejecución aceptable. Por otra parte, DBSCAN demuestra ser la opción más rápida, aunque con una calidad de agrupamiento menor. En cuanto al segundo experimento, la comparación entre los dos algoritmos jerárquicos mediante la prueba de Wilcoxon demuestra que existen diferencias significativas. Se determina que la superioridad previamente observada en UPGMC no se debe a su naturaleza jerárquica, sino a sus características particulares. CURE, con su estrategia de múltiples puntos representativos y contracción, resulta estadísticamente superior en la mayoría de las métricas de calidad y en tiempo de ejecución.

Recomendaciones

A partir de los resultados alcanzados durante el desarrollo de esta investigación, se formulan las siguientes recomendaciones para trabajos futuros.

- Se recomienda explorar la incorporación de otros algoritmos de agrupamiento basados en densidad, como OPTICS, que podría ofrecer una mejor respuesta ante la presencia de clústeres con densidades variables, superando una de las limitaciones conocidas de DBSCAN.
- Dado que el presente estudio se ha centrado en la calidad de la asignación mediante métricas de validación interna, se recomienda experimentar a través de herramientas como BHCVRP con estos dos nuevos algoritmos para ver la calidad de las rutas obtenidas y si se corresponden con lo prometedoras que son las asignaciones, CURE por calidad de las métricas y DBSCAN por su rapidez.
- Por último, se recomienda incorporar estos cambios a la versión en Python de la biblioteca.

Bibliografía

[1] M. E. Coloma Salazar, J. Arzola Ruiz, y C. E. Marrero Fornaris, "La gestión de la distribución y la planificación de rutas en el sector empresarial cubano", *Rev. Univ. Soc.*, vol. 14, pp. 357–368, 2022.

[2] G. B. Dantzig y J. H. Ramser, *The Truck Dispatching Problem. Management Science*, 6.ª ed., 1959.

[3] Clarke, G. y Wright, J.R., *Scheduling of Vehicle Routing Problem from a Central Depot to a Number of Delivery Points. Operations Research*, vol. 12, 1964.

[4] S. Arora y B. Barak, *Computational complexity: a modern approach*. Cambridge University Press, 2009.

[5] K. Braekers, K. Ramaekers, y I. Van Nieuwenhuyse, "The vehicle routing problem: State of the art classification and review", *Comput. Ind. Eng.*, vol. 99, pp. 300–313, sep. 2016, doi: 10.1016/j.cie.2015.12.007.

[6] D. I. G. Mateos, "Desarrollo de un método híbrido para la resolución del MDVRP", *Rev. Esc. Jacobea Posgrado*, 2013.

[7] J. A. G. Bermudez, "Aplicación de técnicas de clusterización como inicializadores y la metaheurística ILS, al Problema de Ruteo Vehículos con Múltiples Depósitos (MDVRP)", Proyecto de grado presentado como requisito parcial para optar al título de Magister en Ingeniería Eléctrica, Universidad Tecnológica de Pereira.

[8] S. Barreto, C. Ferreira, J. Paixão, y B. S. Santos, "Using clustering analysis in a capacitated location-routing problem", *Eur. J. Oper. Res.*, vol. 179, n.º 3, pp. 968–977, 2007, doi: 10.1016/j.ejor.2005.06.074.

[9] Y. Z. Mehrjerdi y A. Nadizadeh, "Using greedy clustering method to solve capacitated location-routing problem with fuzzy demands", *Eur. J. Oper. Res.*, vol. 229, n.º 1, pp. 75–84, 2013, doi: 10.1016/j.ejor.2013.02.013.

[10] A. K. Jain y R. C. Dubes, *Algorithms for clustering data*. Prentice Hall advanced reference series. Englewood Cliffs, NJ: Prentice-Hall, 1988.

[11] K. Khan, S. U. Rehman, K. Aziz, S. Fong, y S. Sarasvady, "DBSCAN: Past, present and future", en *The fifth international conference on the applications of digital information and web technologies (ICADIWT 2014)*, IEEE, 2014, pp. 232–238.

[12] S. Guha, R. Rastogi, y K. Shim, “Cure: an efficient clustering algorithm for large databases”, *Inf. Syst.*, vol. 26, n.º 1, pp. 35–58, mar. 2001, doi: 10.1016/S0306-4379(01)00008-4.

[13] “GitHub - vrpsolutions/BHAVRP: BHAVRP: Componente de software para la asignación de clientes a depósitos en problemas de rutas de vehículos”. Accedido: 21 de enero de 2026. [En línea]. Disponible en: https://github.com/vrpsolutions/BHAVRP

[14] E. R. Aragón, I. T. Pérez, A. R. Suárez, y A. de la C. M. Morales, “BHAVRP: Biblioteca de Heurísticas de Asignación para el Problema de Planificación de Rutas de Vehículos con Múltiples Depósitos”, *Rev. Cuba. Ing. RCI*, pp. 1–13, 2025.

[15] Eric Ramos Aragón, “Incorporación de un módulo de métricas de evaluación para algoritmos de asignación en BHAVRP”, Trabajo para optar por el título de Ingeniero Informático, Universidad Tecnológica de la Habana “José Antonio Echeverría”, La Habana, Cuba, 2025.

[16] S. N. Kumar, “A survey on the vehicle routing problem and its variants”, *Intell. Inf. Manag.*, 2012.

[17] G. Berbeglia, J.-F. Cordeau, y G. Laporte, “Dynamic pickup and delivery problems”, *Eur. J. Oper. Res.*, vol. 202, n.º 1, pp. 8–15, 2007, doi: 10.1016/j.ejor.2009.04.024.

[18] R. Baldacci, M. Battarra, y D. Vigo, “Routing a heterogeneous fleet of vehicles”, en *The Vehicle Routing Problem: Latest Advances and New Challenges*, B. Golden, S. Raghavan, y E. Wasil, Eds., Springer, 2008, pp. 3–27. doi: 10.1007/978-0-387-77778-8_1.

[19] D. Sariklis y S. Powell, “A heuristic method for the open vehicle routing problem”, *J. Oper. Res. Soc.*, vol. 51, n.º 5, pp. 564–573, 2000, doi: 10.1057/palgrave.jors.2600924.

[20] J.-F. Cordeau, G. Desaulniers, J. Desrosiers, M. M. Solomon, y F. Soumis, “The VRP with time windows”, en *The Vehicle Routing Problem*, P. Toth y D. Vigo, Eds., SIAM, 2002, pp. 157–193. doi: 10.1137/1.9780898718515.ch7.

[21] J. R. Montoya-Torres, J. López Franco, S. Nieto Isaza, H. Felizzola Jiménez, y N. Herazo-Padilla, “A literature review on the vehicle routing problem with multiple depots”, *Comput. Ind. Eng.*, vol. 79, pp. 115–129, 2015, doi: 10.1016/j.cie.2014.10.029.

[22] M. Gendreau, G. Laporte, y R. Séguin, “Stochastic vehicle routing”, *Eur. J. Oper. Res.*, vol. 88, n.º 1, pp. 3–12, 1996, doi: 10.1016/0377-2217(95)00050-X.

[23] P. M. Francis, K. R. Smilowitz, y M. Tzur, “The period vehicle routing problem and its extensions”, en *The Vehicle Routing Problem: Latest Advances and New Challenges*,

B. Golden, S. Raghavan, y E. Wasil, Eds., Springer, 2008, pp. 73–102. doi: 10.1007/978-0-387-77778-8_4.

[24] A. Ibrahim, R. Abdulaziz, y J. Ishaya, "Capacitated vehicle routing problem", *Int. J. Res. - GRANTHAALAYAH*, vol. 7, pp. 310–327, abr. 2019, doi: 10.5281/zenodo.2636820.

[25] V. Rebillas Loredo, "The multi-depot VRP with vehicle interchanges", PhD Thesis, Universitat Politècnica de Catalunya, 2018. doi: 10.5821/dissertation-2117-127490.

[26] J. B. Colin, "Diseño de un algoritmo metaheurístico para la solución del Problema de Ruteo de Vehículos MultiDepósito", para obtener el grado de Maestría en Optimización y Cómputo Aplicado, Universidad Autónoma del Estado de Morelos, 2024.

[27] NEO Research Group, "Multiple Depot VRP". Universidad de Málaga, 2019. [En línea]. Disponible en: https://neo.lcc.uma.es/vrp/vrp-flavors/multiple-depot-vrp/

[28] T. R. P. Ramos, M. I. Gomes, y A. P. Barbosa-Póvoa, "Multi-depot vehicle routing problem: a comparative study of alternative formulations", *Int. J. Logist. Res. Appl.*, vol. 23, n.º 2, pp. 103–120, 2020, doi: 10.1080/13675567.2019.1630374.

[29] F. Hamid, Ed., *Optimization essentials: theory, tools, and applications*. International series in operations research & management science, vol. 353. Singapore: Springer, 2024.

[30] M. Jünger *et al.*, "50 Years of Integer Programming 1958-2008: From the Early Years to the State-of-the-Art", 2009, doi: 10.1007/978-3-540-68279-0.

[31] D. G. Mogale, A. Ghadge, y S. K. Jena, "Modelling and optimising a multi-depot vehicle routing problem for freight distribution in a retail logistics network", *Comput. Ind. Eng.*, vol. 207, p. 111315, sep. 2025, doi: 10.1016/j.cie.2025.111315.

[32] E. Erdem, T. Aydin, y B. Erkayman, "AI-Infused Evolutionary Optimization for Multi-Depot Vehicle Routing", *Case Study Beverage Distrib.*, vol. 37, n.º 27-28, dic. 2025, doi: 10.1002/cpe.70431.

[33] F. S. Hernández, "Aplicación del modelo VRP (Vehicle Routing Problem) para la optimización de una red de distribución", Universidad Nacional Autónoma de México, 2015.

[34] I.-M. Chao, B. L. Golden, y E. A. Wasil, "A new heuristic for the multi-depot vehicle routing problem that improves upon best-known solutions", *Am. J. Math. Manag. Sci.*, vol. 13, pp. 371–406, 1993.

[35] F. A. Tillman, “The multiple terminal delivery problem with probabilistic demands”, *Transp. Sci.*, vol. 3, n.º 3, pp. 192–204, 1969, doi: 10.1287/trsc.3.3.192.

[36] B. E. Gillett y L. R. Miller, “A heuristic algorithm for the vehicle-dispatch problem”, *Oper. Res.*, vol. 22, n.º 2, pp. 340–349, 1974, doi: 10.1287/opre.22.2.340.

[37] Y. Shi, L. Lv, F. Hu, y Q. Han, “A Heuristic Solution Method for Multi-Depot Vehicle Routing-Based Waste Collection Problems”, *Appl. Sci.*, vol. 10, n.º 7, 2020, doi: 10.3390/app10072403.

[38] C. Blum y A. Roli, “Metaheuristics in combinatorial optimization: Overview and conceptual comparison”, *ACM Comput. Surv.*, vol. 35, n.º 3, pp. 268–308, sep. 2003, doi: 10.1145/937503.937505.

[39] J. H. Orallo, M. J. R. Quintana, y C. F. Ramírez, *Introducción a la Minería de Datos*. Pearson Educación, 2004.

[40] G. Kesavaraj y S. Sukumaran, “A study on classification techniques in data mining”, en *2013 fourth international conference on computing, communications and networking technologies (ICCCNT)*, IEEE, 2013, pp. 1–7.

[41] Ş. Şeker, A. F. Güneri, G. Ermiş, y İ. Yıldız, “Solving the multi depot vehicle routing problem with assignment algorithm”, en *13th International Research/Expert Conference “Trends in the Development of Machinery and Associated Technology”, Hammamet*, 2009, pp. 225–228.

[42] K. Koutroumbas y S. Theodoridis, *Pattern recognition*. Academic Press, 2008.

[43] S. Salhi y M. Sari, “A heuristic algorithm for the multi-depot vehicle routing problem”, *J. Oper. Res. Soc.*, vol. 49, n.º 4, pp. 424–431, 1998, doi: 10.1057/palgrave.jors.2600526.

[44] JSprit Contributors, “JSprit: A Java-based library for solving vehicle routing problems”. Accedido: 18 de marzo de 2026. [En línea]. Disponible en: https://jsprit.readthedocs.io/en/latest/

[45] L. Perron y V. Furnon, *OR-Tools*. Google. [En línea]. Disponible en: https://developers.google.com/optimization/

[46] F. Pedregosa, *Scikit-learn: Machine Learning in Python*, 12.ª ed., 2011.

[47] Ananda de la Caridad Morales Morales, “Nueva versión de la biblioteca de heurísticas de construcción para problemas de planificación de rutas de vehículos”, Trabajo para optar por

el título de Ingeniero Informático, Universidad Tecnológica de la Habana "José Antonio Echeverría", La Habana, Cuba, 2025.

[48] G. Yañez y I. Torres, "Desarrollo de nuevos modelos para el Problema del Transporte Obrero en CaSVRP", Universidad Tecnológica de la Habana "José Antonio Echeverría", La Habana, Cuba, 2021.

[49] R. D. Hernández, "Nueva versión de la biblioteca de clases BiCIAM para solucionar problemas multiobjetivo", Trabajo de diploma, Universidad Tecnológica de La Habana "José Antonio Echeverría", La Habana, Cuba, 2014.

[50] R. D. Hernández, "Nueva versión de la biblioteca de clases BiCIAM para solucionar problemas multiobjetivo", Trabajo de diploma (Ingeniería Informática), Universidad Tecnológica de La Habana "José Antonio Echeverría", La Habana, Cuba, 2014.

[51] L. Lores, "Operadores de mutación basados en heurísticas de construcción para Problemas de Planificación de Rutas de Vehículos", Trabajo de diploma (Ingeniería Informática), Universidad Tecnológica de La Habana "José Antonio Echeverría", La Habana, Cuba, 2017.

[52] W. Quiala Fonseca, "Agrupamiento de datos desde un enfoque paralelo", *Rev. Cuba. Cienc. Informáticas*, vol. 17, 2023.

[53] A. Patra, R. Khan, y S. Vijayalakshmi, "Customer segmentation and future purchase prediction using RFM measures", en *2022 4th International Conference on Advances in Computing, Communication Control and Networking (ICAC3N)*, IEEE, 2022, pp. 753–759.

[54] M. Ester, H.-P. Kriegel, J. Sander, X. Xu, y others, "A density-based algorithm for discovering clusters in large spatial databases with noise", en *kdd*, 1996, pp. 226–231.

[55] S. Suman, A. Karna, y K. Gibert, "Bootstrap-CURE: A Novel Clustering Approach for Sensor Data—An Application to 3D Printing Industry", *Appl. Sci.*, vol. 12, n.º 4, 2022, doi: 10.3390/app12042191.

[56] W. Wu, H. Xiong, y S. Shekhar, *Clustering and information retrieval*, vol. 11. Springer Science & Business Media, 2003.

[57] J. F. Cordeau, J. Desrosiers, y M. M. Solomon, "The VRP with Multiple Depots: State-of-the-art and New Insights", *Transportation Science*, vol. 36, n.º 4, pp. 278–292, 2002.

[58] M. Friedman, "The use of ranks to avoid the assumption of normality implicit in the analysis of variance", *J. Am. Stat. Assoc.*, vol. 32, n.º 200, pp. 675–701, 1937.

[59] B. Bergmann y G. Hommel, “Improvements of General Multiple Test Procedures for Redundant Systems of Hypotheses”, en *Multiple Hypothesenprüfung — Multiple Hypotheses Testing*, Berlin, Heidelberg: Springer-Verlag, 1988, pp. 100–115. doi: 10.1007/978-3-642-52307-6_8.

[60] S. Holm, “A Simple Sequentially Rejective Multiple Test Procedure”, *Scand. J. Stat.*, vol. 6, pp. 65–70, 1979.

[61] J. P. Shaffer, “Modified Sequentially Rejective Multiple Test Procedures”, *J. Am. Stat. Assoc.*, vol. 81, n.º 395, pp. 826–831, 1986, doi: 10.1080/01621459.1986.10478341.

[62] F. Wilcoxon, “Individual Comparisons by Ranking Methods”, *Biom. Bull.*, vol. 1, n.º 6, pp. 80–83, 1945, doi: 10.2307/3001968.

[63] G. E. Gillett y L. R. Johnson, “Multi-terminal vehicle-dispatch algorithm”, *Transp. Res.*, vol. 10, n.º 1, pp. 47–55, 1976.